

A PROJECT REPORT
ON
BUSINESSLENS - SMART SAAS ANALYTICS DASHBOARD

REPORT SUBMITTED TOWARDS PARTIAL FULFILLMENT OF THE REQUIREMENT
FOR THE AWARD OF THE DEGREE OF

BACHELOR OF COMPUTER SCIENCE ENGINEERING

SUBMITTED BY

Aaryan Dhawan - 22070122002

Abhishek Rajput - 22070122007

Arnav Jain – 22070122030

Himani Arora - 22070122079



UNDER THE GUIDANCE OF

Dr. ANURAHDA PILLAI

Assistant Professor

SYMBIOSIS INSTITUTE OF TECHNOLOGY PUNE - 412115

**A CONSTITUENT OF SYMBIOSIS
INTERNATIONAL (DEEMED UNIVERSITY)**

2025-26



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Pune – 412115, Maharashtra State, India

<https://www.sitpune.edu.in/>

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CERTIFICATE

This is to certify that the Project work entitled “**BUSINESSLENS - SMART SAAS ANALYTICS DASHBOARD**” is carried out by Aaryan Dhawan, Abhishek Rajput, Arnav Jain and Himani Arora, in partial fulfilment for the award of the degree of **Bachelor of Technology in Computer Science & Engineering**, Symbiosis International (Deemed University), Pune during the academic year 2025-2026.

Dr. Anuradha Pillai

Assistant Professor

Dr. Sashikala Mishra

Head, Department of CSE

Declaration

We hereby state that our project work titled — **BUSINESSLENS - SMART SAAS ANALYTICS DASHBOARD** is an accurate record of our work carried out at as requirements for final year project for the award of B.Tech. degree in Computer Science Engineering at Symbiosis Institute of Technology Pune, affiliated with Symbiosis International University Pune, under the guidance of **Dr. Anuradha Pillai** from July 2025 to November 2025.

Aaryan Dhawan
22070122002

Abhishek Rajput
22070122007

Himani Arora
22070122079

Arnav Jain
22070122030

Date: 3rd November, 2025

Certified that the above statement made by the student is correct to the best of our knowledge and belief

Dr. Anuradha Pillai

CSE Department

Acknowledgment

This project's progress and final result involved a lot of support and assistance from many people, and we are incredibly grateful to have this during the completion of our project work. What we did is primarily because of such support and assistance and we wouldn't fail to thank them.

First and foremost, we express our gratitude and praise to Symbiosis Institute of Technology, Pune and Department of Computer Science and Information Technology for giving us this wonderful opportunity to undergo B. Tech Project Work, helping us to learn and attain great experience

We would like to thank our HOD, **Dr. Sashikala Mishra** for her valuable guidance, supervising this work and helpful suggestions.

We owe our profound gratitude and special thanks to **Dr. Anuradha Pillai** for her constant cooperation, valuable guidance, and the generous time she devoted to us. Despite her busy schedule, she patiently listened to our queries, advised us at every stage, and kept us on the right track. Her unwavering support, insightful suggestions, and constructive feedback provided us with direction, encouragement, and motivation throughout the project. We are truly grateful for her continuous assistance, valuable ideas, and critical inputs that helped us successfully complete this work.

We extend our heartfelt thanks to our friends and well-wishers for their support and timely help.

Thank You!

List of Figures

Fig. No.	Title
1	Distribution of Report
2	System Architecture Plan of (Extractive + Abstractive) Hybrid Model
3	Data Flow Diagram
4	Gantt Chart
5	Dashboard
6	Data Upload
7	Revenue Trend Analysis
8	Orders Analysis
9	Customer Metrics Analysis
10	AI Insights
11	Billing
12	Infrastructure Architecture
13	Deployment Architecture (Docker)
14	Deployment Architecture (Docker-Compose)

List of Tables

Table No.	Title
1	Literature Review
2	Software Tools
3	Hardware tools
4	Work-breakdown structure
5	Timeline
6	Author evaluation

ABSTRACT

BusinessLens is a smart analytics platform built in a SaaS style, which aims to serve businesses with real-time analytics, data visualization, and AI-generated recommendations. The system is using a MERN stack for the development and OpenAI is integrated with Stripe for both billing automation and data processing. It equips businesses with a unified and convenient dashboard for evaluating and tracking key performance indicators, revenue, and trends.

Our Project is about the powerful analytics which are to be made less specialized and more understandable through the provision of an intuitive and streamlined interface to aid in data-driven decision making. The users can alter the performance metrics through modular, interactive dashboards and visualizations. The backend is constructed for secure data access and storage, while the frontend, with Tailwind CSS and React, is created for a visually appealing and interactive user experience. Thanks to AI integration, the platform is empowered to identify key insights, provide high-level summaries, and make predictions; furthermore, it translates these technical terms into simple language for strategic decision-making by non-technical users.

BusinessLens supports the authentication of several users, is deployed on the cloud, and uses Docker containerization to provide scalability and adaptability. It is also equipped with BusinessLens that includes Continuous Integration and Continuous Deployment (CI/CD) through Jenkins, thus, it is designed for enterprise-level applications. In addition, the integration of Stripe makes it easy to manage subscriptions and payments, thus, it becomes a fully operational SaaS solution.

In essence, BusinessLens is a SaaS-enabled, AI-powered, analytics-enabled, and comprehensive solution in the modern business environment. It is the less daunting of the two major challenges of turning raw data into actionable real-time information and also allowing organizations to revamp their functional activities, enhance their profit margins, and foster data-centric advancement in the digitally-enabled business landscape.

CONTENTS

1. INTRODUCTION.....	11
1.1 Problem Statement.....	11
1.2 Motivation.....	12
1.3 Objectives	12
1.4 Scope.....	12
1.5 Overview of the report.....	13
1.6 Proposed System.....	13
2. LITERATURE REVIEW	14
2.1 Background.....	17
2.2 Comprative Analysis Of Related Work	18
2.3 Summary of Existing Methodologies	19
2.4 Gaps Identified In Previous Work.....	20
2.5 Research Findings.....	21
3. PLATFORMS/TOOLS/Frameworks USED	22
3.1 Frontend Technologies.....	22
3.2 Backend Technologies	23
3.3 Database and Data Management.....	23
3.4 AI and NLP Integration.....	24
3.5 Deployment and Infrastructure Tools.....	24
3.6 Security and Networking Tools	25
3.7 Development and Collaboration Tools	25
4. SOFTWARE REQUIREMENTS SPECIFICATIONS.....	26
4.1 Introduction.....	27
4.2 Purpose.....	27
4.3 Software Tools	28
4.4 Hardware Tools	28
4.5 Work Breakdown	29
4.6 Functional Requirements	30
4.7 Non-Functional Requirements.....	30

4.8 System Features.....	31
4.9 Assumptions and Dependencies	32
4.10 Data Requirements	32
4.11 System Constraints	33
 5. SYSTEM DESIGN	 34
5.1 System Architecture Plan	35
5.2 Data Flow Diagram	36
5.3 Project Plan and Timeline Chart.....	37
 6. IMPLEMENTATION AND EXPLANATION	 38
6.1 Dataset Processing.....	39
6.2 Algorithm.....	40
6.3 Limitation and Challenges.....	40
 7. SYSTEM IMPLEMENTATION.....	 41
7.1 Module Description	42
7.2 Key Functionalities	43
7.3 Integration Flow	44
7.4 Testing and Debugging.....	44
 8. RESULTS AND DISCUSSION.....	 45
8.1 Author Evaluation	46
8.2 Discussion	48
 9. TESTING	 50
9.1 Testing Strategies	50
9.2 Unit Testing	51
9.3 Integration Testing.....	51
 10. DEPLOYMENT.....	 52
10.1 Deployment Architecture.....	53
10.2 Hosting details	54
10.3 Steps to run.....	55

11. CONCLUSION AND FUTURE SCOPE	56
12. REFERENCES	58
12.1 Datasets	59
12.2 Research Papers	59

1. INTRODUCTION

First of all, in the modern world, businesses require in the first-place real-time insights and analytics to be able to make decisions and analytics to make decisions and also insights to evaluate performances. What is more, having analytics in a dispersed manner becomes a challenge, particularly for small and medium enterprises that are lacking tools. BusinessLens is concentrated on this requirement to introduce a single AI analytics solution that will enable businesses to visualize, evaluate, and gain insights. By using the MERN (MongoDB, Express, React, Node) stack as a development platform, BusinessLens is able to seamlessly blend web analytics technologies and AI to unravel complex business data and present it in an appealing and groundbreaking manner.

On the top of that, the system offers a flexible dashboard that illustrates the key performance indicators (KPIs), trend analyses, as well as AI-driven suggestions for strategic planning. The interface has been created with the help of React, which allows the system to be user-friendly and accessible to users of different groups. The system utilizes Node.js for its backend operations that ensure the safety, effectiveness, and efficiency of data management and use. In addition, BusinessLens through containerized deployment with Docker and continuous integration with Jenkins, ensures that it is not only a theoretical concept but also a viable solution in the real world. BusinessLens modular architecture through new analytic features serves the modern enterprises changing needs and further enhances real-world implement ability. BusinessLens is an AI-powered, advanced analytics, and cloud-ready architecture that acts as a bridge technology in the process of data collection and decision-making between the data layer and the strategic intelligence module. The project, basically, is a demonstration of the empowering role of technology in making businesses smarter and more efficient in the digitally competitive environment which is getting more and more stringent.

1.1 Problem Statement

While organizations work to overcome various business challenges; the significance of data has become indispensable for the analysis and prediction of business trends and performance. In particular, SMEs and other organizations that lack access to sophisticated analytic capabilities struggle to transform data into insights. Those organizations that rely on less advanced data analysis methods find that such approaches are very time-consuming and also that they do not integrate all the business functions. The absence of unified frameworks and tools has resulted in the creation of isolated environments. Pathogenic and untracked systems for data monitoring, invoicing, and reporting that have become complex and thus have a negative impact on organizational efficiency are the consequences of these environments.

The business data environment with advanced analytics tools has changed. BusinessLens intends to occupy the vacant space with a potent, central hub that facilitates the analytic refinements process. This SaaS analytics platform is designed to allow businesses to evaluate their performance metrics, keep a record of business growth, and obtain advanced insights and recommendations all without having to possess advanced expertise. To make the platform billing efficient and to automate the generation of insights, OpenAI and Stripe are being used. In this way, the platform is concentrating on core analytic and insight objectives.

We have also employed the MERN stack to get the maximum benefit of the features such as flexibility, scalability, and cross-platform compatibility so that our project is industry-level deployment-ready and can accommodate various business environments.

1.2 Motivation

Businesses, in this day and age, heavily reliant on data, continue to produce an enormous amount of unstructured data of which these are just some of the forms: reports, financial statements, reviews, and articles. The extraction of actionable insights from such data through manual means is a process that takes a lot of time and is prone to errors. Scattered information and the absence of a unified system that can automatically summarize and interpret complex data leave business executives and analysts in the dark most of the time. Hence, they are unable to make decisions promptly.

The main purpose of building BusinessLens is to close this gap, which was achieved by the integration of artificial intelligence, natural language processing (NLP), and present-day web technologies; thus, the way business information is comprehended and summarized is done automatically. With the help of AI-driven summarization and visualization, BusinessLens becomes the vehicle for faster, smarter, and data-backed decision-making, thus, organizations get to keep their competitive advantage in a digital economy that is changing rapidly.

1.3 Objectives

The major goals of the BusinessLens project include:

1. To architect and execute a corporate intelligence platform driven by AI, capable of automatically generating summaries and performing analyses of the textual business data.
2. To implement hybrid text summarization methods (extractive + abstractive) to produce brief yet informative business insights.
3. To offer an interactive dashboard that intuitively visualizes the user's key trends, reports, and analytics.
4. To guarantee the system can be scaled up, secured, and its components can be used interchangeably by means of a cloud-ready, containerized design with Docker and CI/CD pipelines.
5. To use OpenAI APIs for complex NLP tasks and Stripe APIs for easy payment and billing management.
6. To spread the platform over Vercel (frontend) and Render (backend) with MongoDB Atlas for dependable and secure data storage.

1.4 Scope

The principal goal of BusinessLens has been the design, development, and implementation of a smart web-based analytics dashboard that is adaptable for businesses of various sizes and different domains. The system is designed with the following essential features: data visualization, user authentication, intelligent recommendations, and an automatic billing module. The system delivers analytics in real-time and, due to an interactive dashboard, the users can visualize and interpret the financial, operational, and performance data analytics in a direct and simple way. The team takes the project a step further by employing container-based deployment with Docker and using Jenkins for continuous integration to gain stability and take advantage of scalability in production environments.

BusinessLens AI implementation and robust backend architecture is a demonstrator of how technology can empower data and at the same time facilitate automated decision-making. The undertaking is not confined to analytics as it merely serves as a stepping stone for subsequent initiatives in predictive analytics, business automation, automation, and enterprise-level forecasting, thus making it a scalable and market-ready solution for the advanced digital marketplace.

1.4 Overview of the report

The sections of the report are designed for comfortable reading and viewing by the users. Literature Review is presented in a table format. The paper is divided into sections, which are as follows:

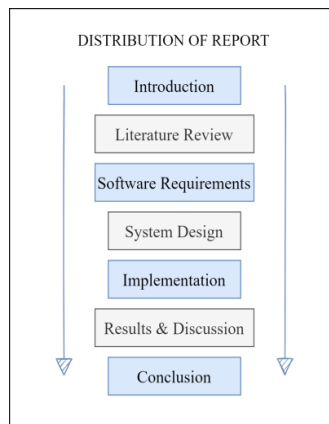


Fig 1. Distribution of Report

1.6 Proposed System

BusinessLens, the proposed system, is a smart, SaaS-based analytics platform that can process raw textual and tabular data to extract summarized insights and visual reports. To achieve this, it uses NLP, machine learning, and data visualization technologies to make the process of decision-making simpler for businesses.

The system's design is based on a MERN stack application — where React and Vite are used for the frontend, Node.js and Express for the backend, and MongoDB Atlas for database management. OpenAI API-powered AI integration is responsible for context-aware summarization and business insight generation, whereas Stripe integration is used for payment handling that is secure and efficient.

By means of a containerized microservice deployment with Docker and automated pipelines via Jenkins, the platform is able to provide smooth updates, continuous delivery, and downtime that is kept to a minimum. BusinessLens is powered by a hybrid summarization engine that combines extractive and abstractive techniques to produce coherent and factual business summaries.

In the end, the proposed system is a single, intelligent, and scalable platform that enables users to input reports, get summaries, visualize trends, and handle billing.

2. LITERATURE REVIEW

Sr. No.	Author	Paper	Model / Category	Description	Result	Inference
1	Awasthi, K. et al. (2021)	<i>Natural Language Processing (NLP) based Text Summarization - A Survey</i>	Survey (Summarization)	Broad survey covering extractive, abstractive, and hybrid summarization methods.	Comparative review of summarization techniques and challenges.	Provides theoretical foundation for understanding core summarization methods used in BusinessLens.
2	Khurana, D. et al. (2022)	<i>Natural language processing: state of the art, current trends and challenges</i>	Survey (NLP)	Comprehensive NLP survey exploring various applications and challenges.	Summarizes key NLP developments relevant to business and AI-driven systems.	Serves as a foundational background for NLP-based insight generation in BusinessLens.
3	Nenkova, A., & McKeown, K. (2012)	<i>A Survey of Text Summarization Techniques</i>	Survey (Summarization)	Classic overview of extractive and abstractive summarization families.	Organizes methods into structured categories and evaluation metrics.	Acts as a conceptual framework for comparing different summarization models.
4	Maybury, M. (1999)	<i>Advances in Automatic Text Summarization</i>	Book (Summarization)	Foundational compilation of early summarization research.	Establishes theoretical background for text condensation approaches.	Lays historical groundwork for summarization evolution.
5	Allahyari, M. et al. (2017)	<i>Text summarization techniques: A brief survey</i>	Survey (Summarization)	Concise review of extractive and abstractive summarization techniques.	Provides simplified categorization and benchmarking overview.	Supports understanding of hybrid methods used in BusinessLens.
6	Kryściński, W. et al. (2019)	<i>Evaluating the factual consistency of abstractive text summarization</i>	Evaluation (Consistency)	Introduces evaluation metrics for factual accuracy in summaries.	Proposes QA-based factuality assessment.	Useful for ensuring reliable insight generation in AI-driven summaries.

7	Cohan, A. et al. (2018)	<i>A Discourse-Aware Attention Model for Abstractive Summarization of Long Documents</i>	Model (Abstractive)	Proposes attention-based summarization for long-form documents.	Improves coherence and factual completeness in lengthy summaries.	Relevant for processing long business reports in BusinessLens.
8	Nan, F. et al. (2021)	<i>Improving factual consistency of abstractive summarization via question answering</i>	Method/Evaluation	QA-based enhancement for factual consistency.	Achieves higher factual accuracy and reliability.	Inspires reliable insight generation by integrating QA evaluation.
9	Di Fabrizio, G. et al. (2014)	<i>A hybrid approach to multi-document summarization of opinions in reviews</i>	Hybrid (Opinion MDS)	Combines extractive and abstractive methods for opinion summarization.	Increases coverage and coherence in summarizing multiple reviews.	Supports hybrid model architecture applied to business data.
10	Jaafar, Y., & Bouzoubaa, K. (2018)	<i>Towards a new hybrid approach for abstractive summarization</i>	Hybrid (Abstractive)	Integrates statistical and neural summarization techniques.	Enhances linguistic richness of generated summaries.	Validates hybrid model selection for BusinessLens pipeline.
11	Sahoo, D. et al. (2018)	<i>Hybrid approach to abstractive summarization</i>	Hybrid (Abstractive)	Uses combination of extractive preselection and abstractive generation.	Improves summary fluency and informativeness.	Reinforces hybrid pipeline efficiency for summarization tasks.
12	Fattah, M. A. (2013)	<i>A hybrid machine learning model for multi-document summarization</i>	Hybrid (MDS)	Combines machine learning with linguistic features for MDS.	Achieves improved summarization accuracy.	Historic evidence supporting ML-based hybrid summarization.
13	Celikyilmaz, A., & Hakkani-Tur, D. (2010)	<i>A Hybrid Hierarchical Model for Multi-Document Summarization</i>	Hybrid (MDS)	Introduces hierarchical summarization using hybrid learning.	Produces summaries preserving document-level hierarchy.	Technical precedent for hybrid multi-layer summarization.
14	Mandal, S. et al. (2017)	<i>A Hybrid Text Summarization Approach</i>	Hybrid (General)	Presents an implementable hybrid summarizer model.	Demonstrates improved precision and readability.	Provides practical blueprint for BusinessLens

						insight summarizer.
15	Kirman, M. et al. (2018)	<i>Hybrid text summarization: A survey</i>	Survey (Hybrid)	Comprehensive review of hybrid summarization approaches.	Classifies hybrid techniques and architectures.	Justifies hybrid selection for BusinessLens summarization strategy.
16	Shi, J. et al. (2022)	<i>Focus-Based Text Summarisation with Hybrid Embeddings</i>	Hybrid (Embeddings)	Combines embeddings with hybrid summarization for focused results.	Improves contextual relevance of summaries.	Encourages embedding-driven summarization for structured insights.
17	Ghanbari Haez, S., & Shamsfakhr, F. (2022)	<i>Extractive text summarisation using Bayesian state estimation of sentences</i>	Method (Extractive)	Uses probabilistic state estimation for extractive summarization.	Achieves balanced coverage and compression.	Can serve as extractive pre-processing layer in BusinessLens.
18	Endres-Niggemeyer, B. (1994)	<i>Summarising text for intelligent communication</i>	Background (Summarization)	Early study on communication-oriented summarization.	Defines goals of meaningful text condensation.	Establishes philosophical base of summarization systems.
19	Benbrahim, M., & Ahmad, K. (1995)	<i>Text summarisation: The role of lexical cohesion analysis</i>	Method (Lexical Cohesion)	Uses lexical cohesion to guide sentence extraction.	Improves coherence of extractive summaries.	Classical reference for cohesion-based extraction techniques.
20	Nenkova, A., & McKeown, K. (2011)	<i>Automatic Summarization</i>	Monograph (Summarization)	Comprehensive overview of automatic summarization principles.	Summarizes historical and modern developments.	Serves as authoritative reference for all summarization frameworks.

Table 1. Literature Rev

2.1 Background

Businesses in the current digital economy are producing and collecting a massive amount of structured data in various data formats such as spreadsheets, CSV files and relational databases. Such data is often of high value as it reflects the company's revenue sources, financial transactions, customer demographics, and operational performance. Unfortunately, in most situations, the actual power of such data is still untapped, for extracting valuable insights is a difficult technical task. Non-technical business users, who make up the majority of decision-makers and are not directly engaged in data-related tasks, usually lack the necessary skills to clean up raw data, unify schemas or carry out complex analytics. Therefore, decision-making gets delayed, insights become fragmented, and organizations find themselves in a state of high dependence on specialized analysts or IT teams for turning raw data into strategic actions.

The conventional method of dealing with structured data has been heavily reliant on manual operations and rule-based systems. For instance, fixing datasets flushed with errors normally entails the writing of scripts, preparing column mappings, or adhering to strict pipelines for format standardization. Although these techniques deliver results in controlled environments, they are inflexible, laborious, and challenging to be scalable across different datasets. Likewise, traditional business intelligence tools are usually developed on the premise that users have preprocessed their data and hence, non-technical users get less opportunities to participate. The gap that exists between raw data and actionable insight is a very loud call to implement more intelligent and user-friendly solutions.

The situation can be changed now radically due to the revolutionary machine learning and artificial intelligence. Transformations-based architectures and large language models (LLMs) which were initially created for natural language processing are equally very efficient in understanding and reasoning with structured data. Models made by training on large datasets of tabular data are not only able to understand the semantics of tables and detect anomalies but also they can generate the relevant insights with very little human guidance. Besides that, the research has moved further in areas like formula prediction in spreadsheets, semantic table interpretation, schema matching and table-to-text generation, all of which are pointing to the eventual automated tabular data understanding.

At the same time, the emergence of no-code and low-code platforms can be viewed as a token of the expanding requirement for democratized data analytics. Business users are becoming more and more vocal in their demand that the technical difficulties should be concealed by the tools and they should be enabled to work with data by means of natural language or user-friendly interfaces. The interaction between these two trends — sophisticated AI models and the need for simple data tools — opens up a possibility to create systems which bring the idea of using the raw business data for strategic insight closer to the reality.

Therefore, it is imperative to comprehend the extent of technological progress by checking out the current research in this domain, discover the best practices and identify the problems that are still unsolved. While most of the research works have made progress in areas that can be considered separately such as cleaning, schema matching, or summarization, only a handful of them have provided an integrated pipeline capable of automatically processing raw, messy datasets from end to end and generating insights in a form that business users can readily apply. Our review acts as a basis for the positioning of our project which utilizes Generative AI and LLMs not just for automating data understanding in a seamless manner but also by going beyond intelligent column detection to generating business actionable insights.

2.2 Comparative Analysis of Related Works

Summarization of 20 different academic research articles was done in such a way that the most dominant themes in these studies became very clear. These themes include survey and foundational works, hybrid model development, and factual consistency improvement.

1. Survey and Foundational Research:

By and large, the studies done by Awasthi et al. (2021), Khurana et al. (2022), and Allahyari et al. (2017) can be considered as exhaustive resources concerning the paradigms of summarization, since they offer detailed taxonomies of extractive, abstractive, and hybrid methods. These pieces of research set the theoretical groundwork for the stages of summarization in a pipeline. BusinessLens takes advantage of these fundamentals in order to organize its AI-driven data insight generation layer.

2. Hybrid Summarization Evolution:

During the years 2010 through 2022, the primary way of thinking shifted from statistically pure methods to hybrid systems that use both linguistic and neural representations. The research work of the likes of Jaafar & Bouzoubaa (2018), Sahoo et al. (2018), and Fattah (2013) on hybrid architectures that present a good balance between coherence and informativeness is well-known.

- o **Observation:** Hybrid models always have better accuracy and readability than single-method systems.

- o **Relevance to BusinessLens:** BusinessLens's insight engine is similarly dependent on hybrid logic - statistical pre-processing is done before neural-language summarization via OpenAI APIs.

3. Factual Consistency and Evaluation Techniques:

According to Kryściński et al. (2019) and Nan et al. (2021), one of the most important aspects of the summaries produced is factual accuracy. The QA-based evaluation methods they developed are the very methods that have motivated BusinessLens's way of cross-checking AI outputs with computed dataset statistics.

4. Context and Long-Document Handling:

Cohan et al. (2018) came up with the idea of using discourse-aware attention models that can deal with very long documents and still keep the main thread of the text. The idea behind this is very similar to the way BusinessLens is trying to solve the problem of analyzing large business datasets while at the same time being able to keep track of the context during insight generation.

2.3 Summary of Existing Methodologies

The combined text summarization and natural language processing (NLP) research form the basis both conceptually and technically for BusinessLens. The major focus of the research has been on both extractive and abstractive summarization techniques, with each having different pros and cons. As per the studies Ghanbari Haez and Shamsfakhr (2022) and Benbrahim & Ahmad (1995), extractive summarization is essentially a method of picking the most relevant sentences or phrases from the source text, thus it is the most factual and coherent summarization. However, the problem is that it normally does not have fluent language and even with the context, it cannot understand which therefore the generated summary is not similar to the way a human would write it.

As compared to that, the goal of research on abstractive summarization models by Cohan et al. (2018) and Kryściński et al. (2019) is to produce summaries which are not just the combination of the phrases or sentences in the source but instead, the models understands, paraphrases and then presents the content in a concise and context-aware manner. To achieve this, such techniques deploy neural networks, attention mechanisms and transformer architectures with the intention of human-like summarization. Nonetheless, the problem of factual consistency still remains the main hurdle, which is the reason why further work such as the one by Nan et al. (2021) suggests using a question-answering system for the improvement of the factual accuracy of the generated summaries.

Between these two extremes, the third—hybrid—models have become a viable option to reconcile the advantages of extractive approaches with those of abstractive ones in terms of linguistic aspects. The researches of Di Fabrizio et al. (2014), Mandal et al. (2017), and Kirmani et al. (2018) exemplify the power of combining statistical, machine learning, and neural methods to address issues related to coverage, coherence, and informativeness. Hybrid systems of this nature empower ones to accomplish better summarization results from the multi-document or semi-structured data thus, they can be considered as the most suitable ones for datasets of the business area that BusinessLens deals with.

Besides these methodological innovations, comprehensive NLP reviews like that of Khurana et al. (2022) and Awasthi et al. (2021) offer a broader and deeper insight into the role which NLP plays in AI-driven applications of today. The surveyed authors agree with the fact that the semantic understanding, contextual embeddings, and evaluation metrics are more and more important in the development of intelligent summarization frameworks. These are the guidelines that BusinessLens follows when it designs its system which is a perfect fit for integrating dataset preprocessing, feature extraction, and AI-powered text generation to provide structured business insights.

To put it briefly, the present summarization techniques supply both the theoretical and practical basis needed for the existence of BusinessLens. The progression in this domain of research from simply extractive methods towards sophisticated hybrid frameworks, is in line with the summarization field moving to more adaptable and context-aware systems. BusinessLens makes use of this hybrid base to turn the structured and semi-structured business data into short, clear insights understandable to people thus, connecting the raw data with the process of making strategic decisions.

2.4 Gaps Identified in Previous Work

While summarization systems have been greatly improved, they still have some gaps, such as the ability to adapt to structured business data, which is mentioned below:

1. Domain Generalization Gap:

Most of the work in summarization has been done with the focus on stories and the like, not really on tables or semi-structured datasets. BusinessLens overcomes this limitation by changing numeric data into narrative insights.

2. Interpretability Limitation:

Earlier models hardly ever accompanied the integration of statistical reasoning with the linguistic summarization. BusinessLens resolves this problem by using an interpretable pipeline — a mixture of data analytics, AI-based summarization, and visualization.

3. Factual Consistency Gap:

Different from QA-based validation that has only been proposed, not many systems have actually adopted it for business summaries in a direct way. BusinessLens is a little effective in handling this issue as it inputs calculated summary statistics into AI prompts.

4. Scalability and Real-Time Feedback:

Most academic summarizers that are talked about in papers take their time and cannot be used for a large amount of data at once. BusinessLens is, however, made for almost real-time processing through an interactive web dashboard.

5. Usability and Visualization Gap:

The previous works have seldom transformed the summaries into visual dashboards. BusinessLens goes further than just summarization to visualization, providing not only KPI charts and graphs but also and text-based explanations at the same time.

2.5 Research Findings

From the comparative analysis and literature review:

• Hybrid Approaches Dominate:

By combining the experimental results of Mandal et al. (2017), Kirmani et al. (2018), and Jaafar & Bouzoubaa (2018) studies hybrid summarization emerges as the best model to achieve the balance of fluency and factuality of the text.

- **Factual Evaluation is Crucial:**

BusinessLens serves as a platform that implements the notion of factual validation layers advocated by Kryściński et al. (2019) and Nan et al. (2021) through the execution of calculated dataset checks preceding AI summarization.

- **Long-Document and Contextual Awareness:**

The studies of Cohan et al. (2018) and Shi et al. (2022) on discourse-aware and embedding-driven models reveal that the need for technologies that can handle large and complex documents is in line with BusinessLens's multi-dimensional dataset inputs.

- **BusinessLens's Contribution:**

The instrument is not limited to just summarizing the research by:

1. **Structured Data Handling** — by merging numeric and textual summarization.
2. **AI-Augmented Analytics** — by linking statistical computation with NLP insight generation.
3. **Interactive Visualization** — by turning the static summaries into an action-capable dashboard.

3. PLATFORMS/TOOLS/Frameworks USED

BusinessLens's development depends on a mixture of contemporary technologies, structures, and instruments that, when combined, result in a solid, extendable, and performant system. Every platform was chosen considering its efficiency, adaptability, and harmony with the project goals of developing an AI-driven, data-centric analytics solution. The following section describes the significant technological innovations that were implemented in the frontend, backend, database, AI integration, deployment, and development lifecycle.

3.1 Frontend Technologies

1. React.js

BusinessLens is powered by React.js, a forefront JavaScript library, which is used to create interactive and component-based user interfaces. React achieves efficient rendering by utilizing a virtual DOM, thus performance is very high even in the case of data-intensive visualizations. Its modular architecture increases the application's life and also allows integration without any issue with visualization and state management tools.

2. Vite

The project has Vite as its build tool and development server. Vite is very fast in providing hot module replacement (HMR), the server can be started instantly, and the production builds are optimized, thus the development workflow is very smooth and efficient. Because of its compatibility with modern ES modules, the compilation is much faster than it would be with a regular bundler.

3. Redux

BusinessLens, in order to maintain application-wide states in a good manner, has incorporated Redux which ensures that the state updates are predictable and that the communication between the components is efficient. In particular, Redux helps in making the user interface more responsive by facilitating the synchronization between user actions, analytics results, and real-time data visualization.

4. Tailwind CSS

With Tailwind CSS as a styling tool, a utility-first approach is adopted, which makes it very easy and fast to create UI prototypes that are consistent in terms of the design language. The main advantage of this tool is that it not only makes the app visually consistent but also, gives responsiveness across different devices.

3.2 Backend Technologies

1. Node.js

The backend leverages Node.js, which is a fast and efficient JavaScript runtime environment made with Chrome's V8 engine. Node's event-driven, non-blocking I/O model is very suitable for a data analytics platform that needs to handle numerous concurrent requests and queries thus, it is able to efficiently manage concurrent requests.

2. Express.js

Express.js is the server-side web framework, which provides a minimal but still very effective frame for routing, middleware integration, and API endpoint management. It makes the clean separation of

concerns possible, thereby increasing the scalability and making it easier to create RESTful APIs for frontend-backend communication.

3. JWT (JSON Web Token)

JWT is employed to secure the communication and authentication whereby it is responsible for issuing and validating tokens during user sessions. This stops any unauthorized access and thus, system security is elevated.

3.3 Database and Data Management

1. MongoDB

The project makes use of MongoDB, a NoSQL document-based database that is capable of storing unstructured and semi-structured data in a very efficient manner. The flexible schema design of the database supports diverse business datasets without the need for a tightly controlled business model, thus it is perfect for dynamic business analytics.

MongoDB's horizontal scalability, high availability, and integration with aggregation pipelines make it an excellent tool for handling large-scale analytical operations.

2. Redis

Redis is being used as an in-memory data store and a caching layer in order to speed up the performance of the system. It keeps the most frequently accessed data such as user sessions, analytics queries, and temporary results and thus, the response times and the load on the database are reduced to a great extent.

3. Mongoose ODM

The Mongoose Object Data Modeling library helps the application to interact with the database more easily by offering features such as schema validation, middleware support, and model-based data access, thus it is the main source of consistency and data integrity in all collections.

3.4 AI and NLP Integration

1. OpenAI API

One of OpenAI's natural language processing models is at the core of BusinessLens intelligent agent. The model is used to analyze, summarize, and interpret business data. Through the API, the system enables conversational insights, whereby users can interact with the system in natural language (e.g., "What was the quarterly growth?") and get the relevant, data-backed answers.

2. Machine Learning & Data Processing Libraries

The use of the statistical computation, data transformation, and preprocessing methods is done via libraries such as Pandas and NumPy (through backend integration or microservices).

3. Charting and Visualization Tools

The visual presentation of the data is achieved through Chart.js and Recharts - the tools thus allowing user-interactive graphs, charts, and dashboards to be created and updated in real-time as the users delve into their analytics.

3.5 Deployment and Infrastructure Tools

1. Docker

Different parts of the app are Dockerized, which means that each piece is packed into a separate container. This is the reason why you can have the same environment in dev, testing, and production. Thus, the frontend, backend, and database as the three services each run in separate containers which makes it a scalable and reliable system.

2. Kubernetes

We can say that Kubernetes is the conductor of these container orchestras, it manages deployment, scaling, and service discovery automatically. On top of that, it offers load balancing and fault tolerance that enable BusinessLens to handle the rise in users seamlessly.

3. CI/CD Pipelines

Continuous Integration and Continuous Deployment (CI/CD) are mechanisms that are used to completely automate the building, testing, and deployment stages. The most significant benefit of this is that the release of new features is done at a much faster pace while the codebase remains stable. There are some tools like GitHub Actions or Jenkins that are used to assure that the DevOps workflow runs without any glitches.

4. Cloud Infrastructure

The platform is designed in a way that it can be deployed on any cloud provider like AWS, Azure, or DigitalOcean and still be able to use the managed services for storage, networking, and load balancing.

3.6 Security and Networking Tools

1. SSL/TLS Encryption

All client-server interaction is protected by end-to-end encryption with SSL/TLS, which is the standard way to ensure that communication is confidential and that the data can be trusted.

2. CORS and Firewall

The Cross-Origin Resource Sharing (CORS) policies are set up in such a way that no unauthorized cross-site request is allowed. The firewall, which is at the next level, uses rate-limiting and intrusion prevention functionalities to provide secure API access.

3. API Gateway and Load Balancer

The API Gateway in the system is responsible for directing the calls from the client to the respective services. It is further supported by a Load Balancer that is capable of spreading the requests evenly to ensure the longest possible uptime and the highest level of stability.

3.7 Development and Collaboration Tools

1. Git and GitHub

The various versions of the code are handled through Git while GitHub serves as the place where the developers can collaborate, track issues, and review the code. The use of branch-based workflows allows for parallel development as well as merges that are under control.

2. Postman

Developers make good use of Postman for testing and debugging their APIs thus they can confirm the endpoints, request parameters, and response formats of their APIs before the actual deployment.

3. VS Code

Among the different options for the developers, Visual Studio Code is the chief integrated development environment (IDE) chosen, as it offers a wide range of powerful extensions, debugging tools, and code management features.

4. SOFTWARE REQUIREMENTS SPECIFICATION

Software Requirements Specification, or SRS in short, is the first document that is created before developing a software. The SRS contains an in-depth review of all the functional and nonfunctional requirements, resources and design details about the software.

4.1 Introduction

The current document is the Software Requirements Specification (SRS) of BusinessLens primarily explains the web application in scope, then lists the functional and non-functional requirements, thus it is a map for developers, designers, testers, and stakeholders that they can use during the entire project timeline.

The stated application will make intelligent, and visually appealing real-time analytics available to every business, no matter what their size is. Customers may use an analytics dashboard for a dynamic, in-depth evaluation of their organizational performance, as well as operational and financial data, to get AI recommendations, and to be able to make their NLP-powered businesses optimized in real-time.

It is a closed-ended business analytics solution, organizational analytics, and a full-stack application that is being developed with the MERN stack (MongoDB, Express, React, Node.js) and will integrate OpenAI for analytical insights and Stripe for billing automation for backend financial analytics and billing.

The BusinessLens SRS document specifies the system's architectural and operational functions and limitations besides describing the interaction of users with the system, data flow and security, system performance, scalability, and the use of Docker containers along with CI/CD and Jenkins.

4.2 Purpose

This document expresses the software requirements for BusinessLens, an AI-powered SaaS analytics dashboard. It will outline the features, performance, and system-level requirements that are expected to be the design, development, and deployment stages of the system. It will be the principal reference that is in line with the needs of the developers, testers, and other people involved in the project.

4.3 Software Tools

<i>Sr. No.</i>	<i>Category</i>	<i>Tool / Technology</i>
1	Frontend Framework	React.js
2	Backend Framework	Express.js
3	Runtime Environment	Node.js
4	Database	MongoDB
5	AI Integration	OpenAI API
6	Payment Gateway	Stripe API
7	Containerization Tool	Docker
8	CI/CD Tool	Jenkins
9	Styling Framework	Tailwind CSS

Sr. No.	Category	Tool / Technology
10	Build Tool	Vite
11	Version Control	Git & GitHub
12	IDE / Code Editor	Visual Studio Code
13	Web Server	Nginx
14	Operating System	Windows / Linux

Table23. Software Tools

All the members of the team used MacOS 12 Monterey and Windows 11 for coding. As the availability of **SSH** and **Bash** commands makes it easier to collaborate.

Managing development on the codebase with multiple developers in the team is made easier by the Distributed Version Control System (DVCS), **Git** and **Github**.

4.4 Hardware Tools

Sr. No.	Hardware Component	Specification
1	Processor (CPU)	Intel Core i5 or higher / AMD Ryzen 5 or above
2	RAM	Minimum 8 GB (Recommended: 16 GB)
3	Hard Disk / SSD	Minimum 256 GB (Recommended: SSD 512 GB)
4	Monitor	15.6" or larger, Full HD display
5	Internet Connectivity	Broadband connection (min. 10 Mbps)
6	Graphics Support	Integrated Graphics (Discrete GPU optional)
7	Keyboard & Mouse	Standard Input Devices
8	Operating System	Windows 10 / 11 or Linux (Ubuntu preferred)

Table 3. Hardware Tools

4.5 Work-Breakdown

Structure:

See table 4 for the work breakdown structure.

Our team members:

- A. Aaryan Dhawan
- B. Abhishek Rajput
- C. Arnav Jain
- D. Himani Arora

Sr.no.	Tasks	Roles and Responsibilities
1.	Brainstorming	A, B, C, D
2.	Planning and Software requirements	A, B, C, D
3.	Project scope and documentation	A, B, C, D
4.	Literature Review	A
5.	Designing	B
6.	Developing functional specifications	C
7.	Implementation	D
8.	Blackbook	A, B, C, D
9.	Research Paper	A, B, C, D

Table 4: Work-breakdown structure

4.6 Functional Requirements

Functional requirements refer to the services that model has to offer to the user of the model/software. It doesn't refer to the quality of those services.

- Users should be able to register, securely log in, and log out via email authentication. Secure session management should be in place to ensure safe access.
- The system should support differentiated role access to the system where normal users and administrators are distinguished. Admins will be capable of account management and supervision of the system activities in addition to the extended privileges.
- Users should be able to present business data files in CSV and JSON formats to the system for analysis and processing. Validation of the data will be done before analysis.
- The system should organize and present the incorporated data in a logical pattern within a personal dashboard using a table and graphic interface.
- The system will use the OpenAI API for the integration of AI-driven tools in the system..
- The system should calculate and present the real-time KPIs of revenue trend changes, growth rates, customer activity statistics, and sales analytics.

- To enhance comprehension and insight of intricate data, the system should provide interactive graphics, including, but not limited to, graphs, charts, and heatmaps.
- The system will use Stripe for the management of subscription recurrent billing, payment processing, and invoice issuance.
- The system will provide email updates.

4.7 Non-Functional Requirements

These are the qualities related to a software service offered to the end user. Some non-functional requirements of our software are:

• Performance Requirements

The system should be able to respond to the user operations within 2-3 seconds for the major interactions: log-in, dashboard rendering, and insights generated by the AI. The back end should be able to handle 1,000 simultaneous user sessions and still be responsive.

• Scalability

The system should be able to scale both vertically and horizontally and support the deployment of dockerized containerized environments and service scalability through orchestration tools.

• Reliability and Availability

The system should be able to achieve 99% downtime over a month (30 days). Besides that, the system should also be able to recover smoothly from minor interruptions, for example, the automated service restarts after a crash, and self-reboot from service downtimes.

• Security Requirements

The system should use HTTPS for the transmission of all secure user data. Systems should protect passwords by encryption, session control by JWT, and mitigations for attacks XSS, CSRF, and SQL/NoSQL injections.

• Maintainability

The program should be able to modularity, inter alia, by making sure the clean division between the front end and the back end, and the external services. Also, the codebase should be consistent with the common standards so that maintenance and upgrading can be done without difficulty in the future.

4.8 User Requirements

User Requirements detail the anticipated features, communication, and working goals from the users' point of view. As BusinessLens is a SaaS-based analytics platform, it is meant for a wide range of users, which include small business owners, data analysts, and financial managers. Knowing what users expect is instrumental in matching system functionalities and thereby achieving good usability and user satisfaction.

User Categories

1. Administrator (System Owner):

- o A person who is responsible for managing the backend system, controlling user access, subscription policies, and overall system maintenance.
- o Has the ability to access completely all the dashboards, logs, and configurations.
- o Monitors system status, API usage, and manages OpenAI and Stripe integrations.

2. Registered User / Business Analyst:

- o The primary end-user of the platform.
- o They upload data, request insights, view analytics dashboards, and manage billing subscriptions.
- o They are provided with the means to view AI-generated insights, highlight trends, and print reports for business decision-making.

3. Guest / Trial User:

- o A user with limited access for demonstration purposes.
- o They are given the permission to have a look at the sample dashboards but cannot upload their personal data or use paid services.

User Goals and Expectations

Users are looking for the system to make the complicated process of analyzing business data by means of automation and AI assistance easier. The principal objectives are:

- **Ease of Use:** The interface ought to be self-explanatory with the help of the visuals of the data being very clear.
- **AI Insight Generation:** Users should obtain significant results without requiring a data scientist of expert level.
- **Speed and Responsiveness:** The dashboard has to be capable of quickly visualizing data and providing AI insights in a few seconds.
- **Data Security:** Users want to be sure that the data they upload is kept private, and no one unauthorized is allowed to access it.
- **Billing Transparency:** Subscription and payment-related actions have to be put forward clearly, together with the simple cancellation or renewal options.
- **Availability:** The system ought to be at the users' disposal 24 hours a day, 7 days a week with a very short or even no period of unavailability in order to be convenient for users from different parts of the world.

User Scenarios

- **Scenario 1:** A startup founder figuring out the non-selling products with the help of AI insights decides to make the sales data public.
- **Scenario 2:** A marketing analyst by looking at the revenue generated from different regions is able to keep a record of the monthly performance.
- **Scenario 3:** A finance manager purchasing a premium plan in order to get automated insights and predictive trend reports.
- **Scenario 4:** The admin keeps track of the system usage, API consumption, and billing webhooks management for users who have subscribed.

User Interaction Requirements

- The front-end should show up-to-date and interactive dashboards that can be used with different devices (desktop, tablet, mobile).
- Without a doubt, success/error messages must accompany every user operation (upload, insight generation, billing) to confirm the result.
- To improve the user experience, the system can make use of some visual feedback means such as loading animations, tooltips, and toast notifications.
- The profile area provides the user with the ability to personalize (theme selection, notification preferences, subscription plans).

4.9 System Features

The system consists of components that are intimately connected and carefully designed to work together in achieving users efficiently managing their business data. The primary objective is to transform raw data into profitable insights through the use of artificial intelligence and data visualization.

Handling data, which begins with the uploading, is undoubtedly one of the major functions of the system. Users can upload data files that are automatically checked and cleaned by a pipeline. The system detects missing data, data of wrong types, or errors and, generally, it fixes them if they are not already done so before the analysis. Therefore, the outcomes will be accurate and trustworthy.

After the feature list comes the AI-powered insight generation. The moment data is fed to the system, it immediately asks the AI model for the ways to produce the summaries, patterns, and suggestions with the highest level of significance. The insight language is extremely simple and, besides that, it is written in one common language thus a non-technical user can understand the results on his/her own.

The dashboard, actually, is one of the main features that facilitate the user with the interactive charts and visualizations. Consequently, the user is empowered to interact with his data in a dynamic manner, check the metrics by category, date, or product and thus grasp business trends. In addition, the user can save or export the reports for the purpose of viewing them offline.

Data safety measures that are based on authentication, authorization, and encrypted communication have the purpose of protecting data. Billing and subscription modules are in charge of payments that are made through the integrated payment gateways in a secure way. Hence, alongside system monitoring and maintenance via the admin panel, managing users and keeping track of API activities become not only easy but also efficient.

These features are all interconnected to provide a system that is robust, dependable, and user-friendly simultaneously, it has the capacity for higher loads and is scalable for further enhancements and updates.

4.10 Assumptions and Dependencies

An unconcealed peculiarity of the instrument is the explicit mentioning of various assumptions that the design and the running of the system depend on. As a matter of fact, an assumption is made that the user is going to have an uninterrupted internet connection as the system is available through a web-based platform. What is more, some technical and computer skills should be possessed by the users so as to be able to upload their files and understand the dashboards. In particular, it is emphasized that the validity of the datasets, i.e., files, is the most crucial factor among other things.

Furthermore, one more crucial assumption is that the system can always use the necessary and reliable third-party APIs such as OpenAI for generating insights and Stripe for billing purposes. If there is a temporary discontinuation, the features that the system offers may not be fully available. Similarly, a hosting place such as Render, Vercel, or the user's local machine is expected to be in a working condition and accessible.

The system also relies on external technologies and frameworks, such as Node.js, Express.js, React.js, and MongoDB. These are the major contributors to the system's overall functionality. Without the team's knowledge, the system may be slow due to updates, version changes, or incompatibility in these dependencies.

Additionally, the platform utilizes Docker containers for deployment consistency and secure email services for sending notifications. In order to guarantee the uninterrupted service, these dependencies should be up-to-date and properly managed.

Lastly, the assumptions and dependencies outline the device's technological and operational foundation that it is based on, thus, being aware of the factors that influence its uptime and performance.

4.11 Data Description

Data is the heart of the system which is responsible for all the analysis and AI-generated insights. Usually, the data that is uploaded is to a large extent, the content of the business in the form of sales records, revenue, product categories, and the dates of transactions. The system takes the data in formats such as CSV or Excel that are compatible with most business tools.

After the user uploads the file, the system goes through it by not only verifying the layout but also identifying any missing or incorrect values and, finally, cleaning the data to make it ready for the subsequent analysis. This preparatory step is mandatory if the quality of the results is to be maintained. The cleaned data thus obtained is stored in a secure manner in a database, which ensures that the data can be retrieved at any time for reanalysis or the generation of reports.

Normally, each dataset comprises numerous fields like date, region, product name, quantity sold, and total revenue. These fields equip the AI model with the capability to recognize the patterns and generate the insights that are highly relevant to the user's business scenario. The processed data is displayed in diagrams and graphs that users can easily understand to identify the trends and outliers.

Moreover, the data description tells about the ways the data is handled and secured. The system that allows data upload guarantees the safety of the data as well as its association only with the respective user account. Data is kept in a safe place using the latest encryption methods and even if someone tries to access it without permission, the data is not accessible to him/her.

To sum up, the data description is the documentation that lays out the data's character, its different components, and processes of dealing with it in the system. By doing so, it not only shows that the system is capable of handling datasets from the real world in a safe, reliable, and effective way but also that it can perform the analysis with high accuracy.

4.12 System Constraints

Every software system has limitations that define the system's environment, its design, the implementation, or development, and the performance. These limitations can be of a different nature, e.g., technical, operational, or resource-based. As far as this system is concerned, one of the significant technical constraints is the reliance on external APIs. Because OpenAI and Stripe services are third-party integrations, any time OpenAI or Stripe APIs are down, their limited availability will, therefore, affect the system's performance.

Moreover, the hosting and deployment issue has been raised as another technical constraint. The system is tailored to work efficiently on platforms like Vercel or Render, nevertheless, these platforms might have imposed limitations such as bandwidth, memory, or execution time restrictions. Also, the performance in a local environment may vary depending on the user's hardware and internet connection.

Operational constraints are also on the list comprising, for example, the limited number of concurrently user sessions during testing or academic evaluation. The system has undergone optimization for a moderate amount of data; however, when processing a very large dataset, the system may slow down considerably. To add to that, security constraints have been considered as well; encryption and token-based authentication have been put in place, but more elaborate methods like multi-factor authentication or intrusion detection have not been part of the current project scope.

The resource constraints are mostly about time, manpower, and infrastructure. The system was created following a fixed academic timeline, so the features like extensive analytics, predictive forecasting, and large-scale testing have been deferred to future versions. In spite of these restrictions, the system manages to perform equally well in terms of accuracy, speed, and usability.

Essentially, the system constraints section is about acknowledging the limits that form the project's background and making sure that these limitations are properly documented as a base for the next steps of improvement and optimization.

5. SYSTEM DESIGN

5.1 System Architecture Plan

BusinessLens's System Architecture is a tiered, modular architecture tailored to maintain its expansion, dependability, accuracy of data, and quickness of performance in real-time with various users and surroundings. The configuration is a hybrid microservices-based one which pivots the four major functional layers—frontend, backend, data management, and cloud infrastructure. It also allows them to interact with each other via RESTful APIs and secure network.

The React.js with Vite is used at the frontend level for fast rendering and optimized performance. Users are offered by this interface with a responsive, easy to understand, and analytics-focused experience. The principal elements such as the dashboard, insights panel, and the data visualization instruments (which are made use of by the likes of libraries such as Recharts or D3.js) enable the real-time exploration of the uploaded business data. The Redux Toolkit is implemented for the global state management effectively, thereby it is data visualization and UI that are consistent even when there are a large number of API calls.

The backend layer, comprising of Node.js and Express.js, is the driving force of the system and hence the brain. It sees user authentication, request routing, data preprocessing, and orchestration. Furthermore, the backend features a link to OpenAI APIs for natural language understanding, thus allowing the system to create business insights in a conversational way. Along with that, middleware parts make sure that input validation, security filtering, and optimized request handling are done. By using Docker, the backend is made up of containers which provide uniform deployment across different environments and also makes CI/CD workflows simpler.

The data storage layer is based on MongoDB that acts as the main data store, and Redis is that provides an ultra-fast in-memory cache. MongoDB is the one to hold on to all the permanent data - user profiles, uploaded datasets, processed analytics, and insights - in flexible collections that can be scaled horizontally through sharding. Redis is the one that makes up the difference by providing a cache for the most frequently executed queries, therefore the response time is very short and the system's performance during the peak hours is highly improved.

The security layer does everything needed to guard data and system integrity against different threats. The system is very strict in enforcing JWT-based authentication, which infers that access to the data pipelines is only allowed to verified users. Transport security is assured via SSL/TLS encryption while CORS policies and input sanitization are employed to prevent cross-site attacks. Other measures like API rate limiting, firewall settings, and role-based access control (RBAC) are there to give a strong protection against the threats of abuse or unauthorized access.

The infrastructure and deployment layer uses Docker containers together with Kubernetes orchestration in a way that the system can operate without any faults. Load balancers are distributing the incoming requests to the different instances on a dynamic basis thus the availability is ensured even during a heavy traffic period. CI/CD pipelines are the ones that do the building, testing, and deploying stages automatically and they are supported by the tools such as Jenkins or GitHub Actions. They open the road for smooth and error-free rollouts. The use of cloud storage and backup methods are the steps that have been taken to make sure the data is secure against any disaster while the monitoring tools such as Prometheus and Grafana are constantly keeping an eye on the system performance, network health, and anomaly detection.

The communication layer at the very end connects the different components with each other by the means of the RESTful APIs and WebSocket protocols. Thanks to this, client and server can be kept in sync almost without any delay, the events can be handled asynchronously, and the updates are of

course in real-time. Apart from that, the modular architecture is a kind of a safety net that allows the individual services (frontend, backend, or database) to be updated or scaled independently without the system's overall stability being affected in any way.

The BusinessLens architecture, in sum, is an exemplary model of a tightly interworking yet weakly connected system that manages the equilibrium between the two attributes of the system i.e. adaptability and stability. Not only does it cater to the present data analysis needs, but it is also ready for the future AI incorporation and cloud scaling, thus, it becomes a solution that can be used at the enterprise level for the large-scale business intelligence systems.

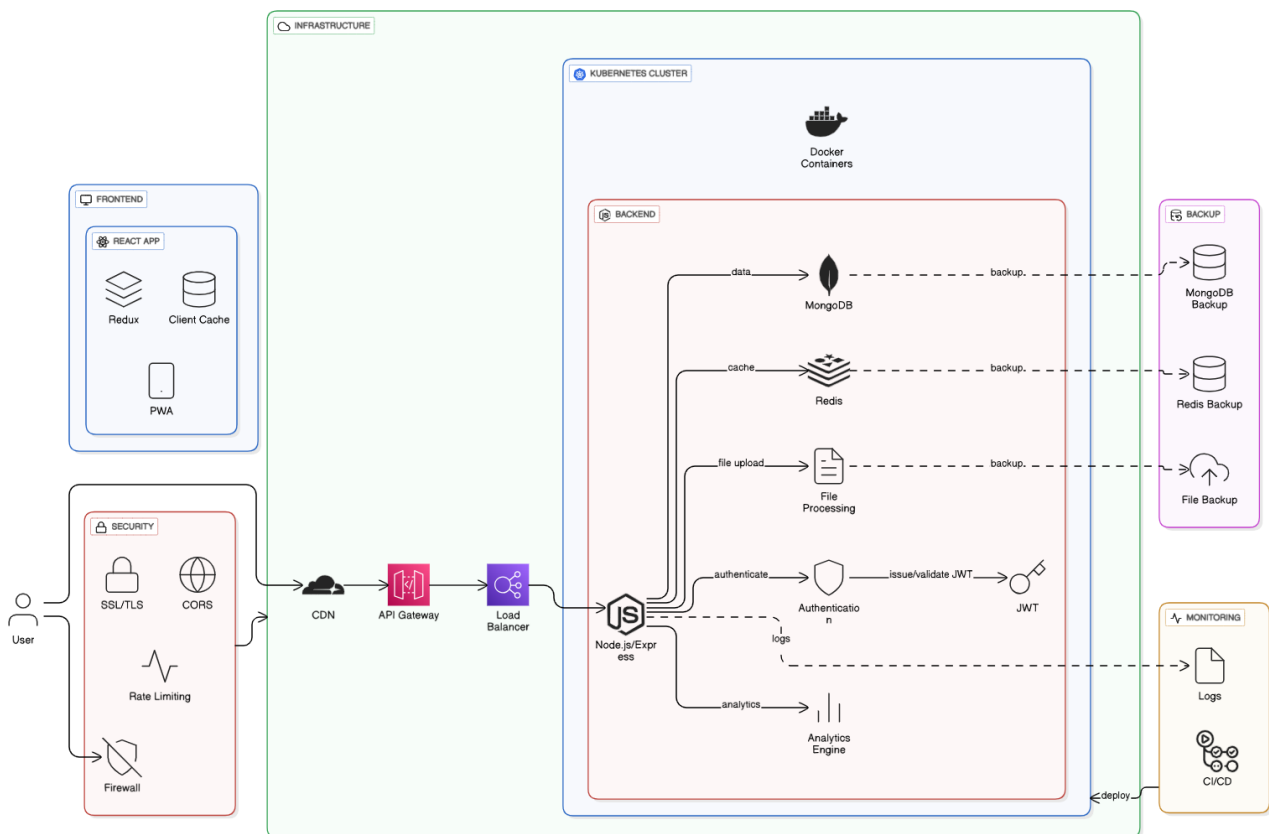


Fig 2. System Architecture Plan of (Extractive + Abstractive) Hybrid Model

5.2 Data Flow Diagram

The Data Flow Diagram (DFD) of BusinessLens is a detailed map of how data moves within the system, starting from the user input and going all the way to the creation of insightful business data. It shows that data is flowing in an orderly manner through different modules, with each module having its own clearly defined operations of input, processing, and output.

Everything starts with the User Module. People use the application through the web-based interface. Users, for example, register accounts, log in, upload datasets or request analytics. In response to these actions, requests are sent to the Authentication Process that is in charge of verifying credentials using JWT tokens and assigning roles (admin, analyst, or viewer) based on the information in the tokens.

After a user is verified, he/she is allowed to proceed with the Data Upload Process. Several data sets examples are sales, inventory and financial data, which are verified for their conformity to formatting rules, schema, and file types. Additionally, data preprocessing is done for instance, missing-value

handling, outlier detection). The system keeps the data in a staging area that can be either in MongoDB or a cloud storage such as AWS S3. The primary purpose of this is to make sure that the raw data is available while the processed versions are used for analytics.

The data that passes the validation stage is then transferred into the Processing and Analytics Module which is considered to be the computational core of the system. The backend performs the transformations, normalization, aggregation, and computations before the data is taken to pipelines. The pipelines utilize OpenAI or locally trained models for getting the insight, trend forecasting, and anomaly detection. The intermediate results are stored in Redis for a quick retrieval.

After that, the Insight Generation Process refers to the cleaned data that is changed into quite comprehensible textual, pictorial, and suggestion formats and these are routed to the frontend for the user's eye. Visiting the user might be an ordered set of concepts such as income change, competition of the performance, or market predicting variables, each of which is dynamically displayed within the dashboard through React components. Meanwhile, the Monitoring and Logging Module records all the user actions, API requests, and computation logs, it constitutes an audit trail for fixing bugs, auditing, and tuning the system's effectiveness. Besides that, the data flow is a strong argument for the presence of the two-way routes for the instant feedback mechanism: when users adjust their query or filter parameters, the frontend dispatches the updated requests that lead to new computations, thus, an interactive loop is preserved.

The Data Storage Process is holding different logical collections for raw data, processed data, and user analytics reports. The separation of these collections allows for modular querying and makes the backup process easier. There is a periodical synchronization that serves the purpose of saving the most recent analytics and insights securely, and at the same time, keeping a cache for user sessions at high-speed. At the end of the flow comes the Output Module which is concerned with displaying the results through dashboards, downloadable reports, and data export options (CSV, PDF). Besides, users are enabled to interact with these outputs by means of chart filters, time-based comparisons, or AI-assisted Q&A—where the system understands natural language queries and provides the answers in the form of contextual business intelligence.

This is how the system guarantees transparency, modularity, and good use of resources at different stages of the system lifecycle. Every conversion—be it from data ingestion to visualization—is identifiable and has been fine-tuned to be of the shortest possible duration. Apart from ensuring analytical accuracy, the DFD is also in harmony with BusinessLens's central objective, i.e., providing intelligent, real-time insights as a means of empowering business decision-making.

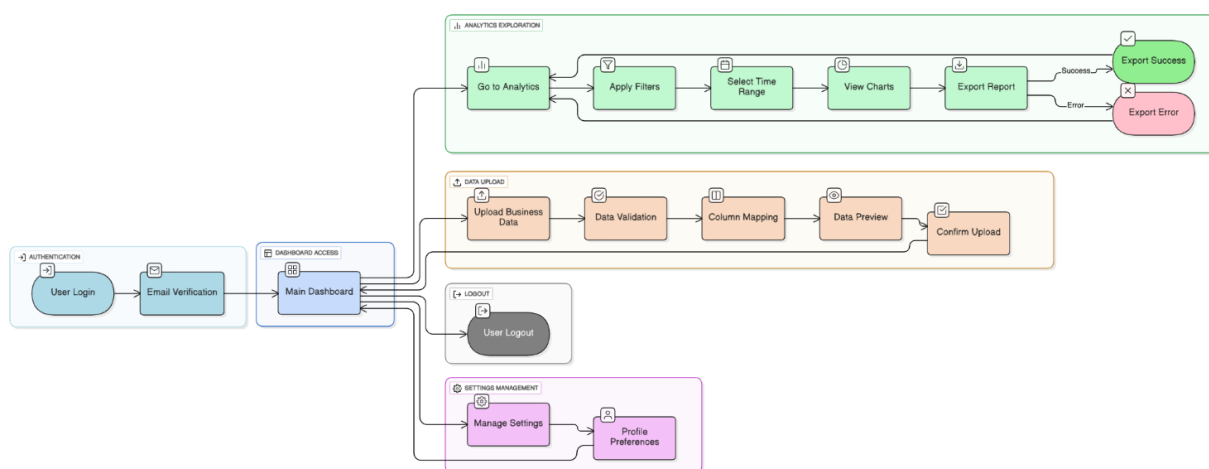


Fig 3. Data Flow Diagram

5.3 Project Plan and Timeline Chart

Sr. No.	Project Phase	Duration	Timeline
1	Brainstorming & Idea Finalization	5 days	1st – 5th July
2	Requirement Gathering & Literature Review	7 days	6th – 12th July
3	System Design	10 days	13th – 22nd July
4	Environment Setup & Tool Configuration	5 days	23rd – 27th July
5	Frontend Development	10 days	28th July – 6th August
6	Backend Development	10 days	7th – 16th August
7	API Integrations (OpenAI, Stripe)	7 days	17th – 23rd August
8	Testing & Debugging	8 days	24th – 31st August
9	Deployment & CI/CD Setup	6 days	1st – 6th September
10	Documentation & Final Report	7 days	7th – 13th September

Table 5. Timeline

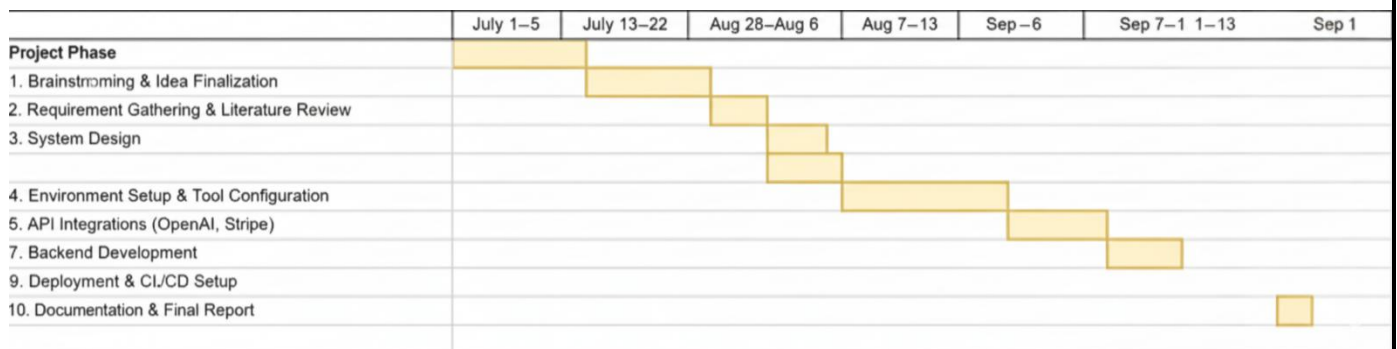


Fig 7. Gantt Chart

6. IMPLEMENTATION AND EXPLANATION

6.1 Dataset Processing

The dataset processing pipeline of a system is designed to achieve the intake of both structured and semi-structured business data such as sales reports, customer data, and performance metrics. Basically, this is a point of transforming data into an AI-powered insight generation compatible format by checking their consistency, validity, and interpretability.

Step Involved:

- **File Upload and Validation**

Users may upload CSV or Excel files. Meanwhile, the backend which employs Multer for handling file uploads also performs file format, size, and encoding checks before the files are permitted for further processing.

- **Data Cleaning**

The data that have been uploaded are transformed into a format that is appropriate for the subsequent work with the help of data libraries of Pandas (Python) and Node.js. The missing data are filled by the method (mean, median, or zero-filling) that has been chosen. Unnecessary columns are removed and data types are converted into standard ones (numeric, categorical, date-time).

- **Schema Detection and Column Classification**

The AI system detects the schema of your computer files and simultaneously makes use of the local files to understand the column headers for the purpose of finding their meaning (e.g., "Revenue," "Customer Name," "Transaction Date"). Therefore, it is feasible to link up the standard business categories such as Sales, Expenses, and Time Period.

- **Normalization and Transformation**

In most cases, the values are normalized so that the same data can be understood in the same manner. As an example, monetary and date formats are standardized. Techniques for outlier removal are employed to eliminate the noise.

- **Feature Extraction**

New features like total revenue, average transaction value, or customer retention rate can be generated. Such features become the background of AI-driven insights.

- **Storage and Accessibility**

After the data have been cleaned and processed, they are still maintained in MongoDB collections not only for permanence but also for quick retrieval. Due to privacy reasons, each dataset is linked with a user ID, thereby permitting modular access control as well.

The preprocessing moment here is somewhat like a confirmation that the AI-generated insight module will get data which are not only well-organized but also rich in content, and this is an extremely important factor for top-quality leading business insights.

6.2 Algorithm

The primary algorithm that powers BusinessLens is fundamentally an AI-based insight generation system combined with conventional data summarization methods. Essentially, the system is a hybrid pipeline that sequentially performs preprocessing, feature detection, and language generation.

Algorithm Steps:

1. Input Phase:

- Receive a dataset uploaded by a user (CSV/XLSX)
- Perform data validation and cleaning of the dataset.

2. Preprocessing Phase:

- The system identifies the data types of columns by using pattern matching and statistical inference.
- It calculates summary statistics such as mean, median, max, min, and standard deviation for every numeric column.

3. Feature Analysis Phase:

- The system finds the relationships between different metrics (e.g., sales vs. expenses) through correlation analysis.
- It also identifies anomalies or trends via regression-based pattern detection.

4. AI Insight Generation:

- The OpenAI API is provided with the cleaned dataset alongside computed summary statistics to generate insights.
- A structured prompt is not static but is generated dynamically each time, describing the dataset context and requesting interpretive insights in natural language.
- Example prompt: "Analyze the following dataset summarizing monthly sales and expenses. Identify major trends, growth periods, and anomalies, and produce a short business summary."
- The AI model comes up with a report in textual form that contains insights such as growth trends, correlations, or performance bottlenecks.

5. Visualization Phase:

- The frontend dashboard uses Recharts to visualize both the summarized data and the generated insights.
- Bar graphs, pie charts, and line graphs are some of the ways that key metrics are presented for easy understanding.

6. Output Phase:

- Final outputs include:
 - Numerical summaries (KPIs)
 - AI-generated textual insights
 - Graphical visualizations

Pseudocode Representation:

```
Begin
  Upload dataset D
  Validate D
  Clean D → D_clean
  Extract features F from D_clean
  Compute summary statistics S
  Generate prompt P using (D_clean, S)
  Send P to AI_Model → Insights I
  Visualize (F, S, I) on dashboard
End
```

6.3 Limitations and Challenges

1. Dependence on Third-Party APIs

One of the system's limitations is reliance on OpenAI for AI-based insights and Stripe for billing. Every system's core functions will be affected if there is system downtime, quota limits, or changes to the pricing and terms of the service.

2. Limited Dataset Interpretation

The AI system attempts to generate insights from business data, but it can only be as invaluable as the input data discloses. Inconsistent and unstructured datasets will negatively impact the quality of insights and the usefulness of insights generated.

3. Resource Constraints

The available academic infrastructure and deployment budget were the main reasons for developing and testing the platform on local or small-scale cloud environments. The project timeline also meant large-scale performance testing, load balancing, and real-world stress simulations were not possible.

4. Scalability Testing

The system is made to support horizontal scalability using Docker, but actual multi-server deployment and orchestration of cloud services like AWS, and Kubernetes were not executed. Time and resource limitations were the primary reasons.

5. Security Scope

Security was Implemented using JWT authentication but industry level security like audit traits and penetration testing were beyond the scope this project.

7. SYSTEM IMPLEMENTATION

The implementation phase is the stage where the theoretical design is changed to an actual operational and functional product. For the BusinessLens initiative, this phase revolved around software module integration, backend logic coordination with the frontend, API setup for AI-driven analytics, as well as the relieving of deployment and database operations. The implementation follows modular development principles; thus, it is possible to develop, test, and integrate concurrently without system conflicts.

7.1 Module Description

BusinessLens is a comprehensive system that achieves the full functionality of the platform by classifying the system into different key modules, each having a unique set of features.

1. Authentication Module

Firstly, it is the module that deals with the verification of user identities as well as secure access. Features like user registration, email verification, password encryption, and token-based authentication using JSON Web Tokens (JWT) are some of the facilities available in it. The ultimate objective is to certify the accessibility of the system to only those users who have been validated and authorized.

2. Dashboard & Analytics Module

The main interface that the users engage with is the dashboard & Analytics module. It visually presents the business insights that have been summarized through the use of charts, tables, and KPI cards. The users here have the liberty to filter the data, select custom timeframes, and generate AI-driven analytics reports. The dashboard makes a direct connection with the backend APIs to retrieve data from MongoDB as well as the OpenAI-based summarization services.

3. Data Upload & Processing Module

Essentially this module is designed to empower the users to upload structured business datasets such as CSV or Excel files. The data that is uploaded goes through a validation phase where any incorrect or missing values are flagged. Afterward, the system executes the tasks of column mapping and data preview, and then data ingestion into MongoDB takes place. Backend scripts are used to clean, standardize, and tag the uploaded data so that it can be used for downstream analytics.

4. AI Insight Generation Module

This module is equipped with the OpenAI's API, which helps it to understand the uploaded data and give textual and contextual insights. It is very efficient in the automatic identification of the patterns, trends, and anomalies, and then it compiles a summary of these in natural language. This is the module that constitutes the intelligent layer of BusinessLens that connects business data with AI interpretation.

5. Reporting & Export Module

This module gives the users the ability to save and export the visualized analytics in PDF, Excel, or CSV reports. The system also has a provision for the automated report scheduling feature through which the recurring reports are generated and sent to the stakeholders via email. The success or failure of the export is kept under observation through facsimiles or logs.

6. User Settings & Profile Module

This module opens up a world of customization options that may include the management of preferences, changing of account details, or adjustment of the dashboard layout. The device substantially improves the personalization and usability aspects by allowing for a diverse range of users to have an input.

7. Admin Monitoring Module

This module is mostly concerned with the administrators and it keeps track of API health, user activity, and usage analytics. It cooperates with the logging services to locate anomalies, aborted transactions, or areas of suspecting patterns by the user.

All of the modules are in constant communication with each other through well-defined RESTful APIs, which is a way of strong interaction and less dependent on each other for the ease of maintenance and enlargement of the system.

7.2 Key Functionalities

The *BusinessLens* platform implements several key functionalities that drive its overall performance and user experience:

1. Secure User Authentication

- An enhanced login procedure is done by email verification and the usage of JWT-based session management.
- The token refreshing is done in the background, and the access control is based on roles (Admin/User).

2. Dynamic Data Upload Pipeline

- The upload is performed by a drag-and-drop method and the validation takes place instantly.
- Moreover, the system automatically recognizes the schema for column mapping.
- The methods of error handling are designed to ensure that there is no missing or incompatible data.

3. AI-Driven Analytics Engine

- Text-to-Insight: OpenAI API is used by the system to transform the input data into narrative summaries.
- Trend Prediction: it employs the statistical methods to forecast very short-term trends.
- Contextual Querying: users can provide natural language command such as "What is the quarter with the highest revenue"?

4. Visualization and Reporting

- Users of the platform can interact with the system through various chart types (bar, pie, line, area) that have been implemented using Recharts and Chart.js.
- The user is allowed to select time periods and use filter options for their data.
- Besides that, there is the option to create reports that take up less space and are saved automatically in different file formats (pdf, excel, csv).

5. Real-Time Synchronization

- The changes to datasets or analytics that have been made locally or remotely are instantly reflected in the dashboard through the use of Redux state management and WebSocket-like updates.

6. Error Handling and Logging

- Global error handlers on both client and server sides.
- The log management system was set up together with an alerting system for API call failures or data mismatches.

7. Scalability and Performance Optimization

- The application of the microservices pattern was containerized with Docker.
- Backend requests were load balanced. The data fetching process was accelerated through caching based on Redis.

7.3 Integration Flow

The BusinessLens integration flow can be considered as the different layers of the system frontend, backend, AI API, and database that are always updated and interconnected.

1. Frontend ↔ Backend Integration

The React frontend uses REST APIs of the Express backend through Axios. Authentication through JWT tokens is done in all requests. When data fetching or posting events are performed, Redux actions are called, while asynchronous thunks are those which handle API call results and UI state updation.

2. Backend ↔ Database Integration

The Node.js backend is using Mongoose ODM for its operations with MongoDB Atlas. CRUD operations have been performed on business datasets, user profiles, and transaction logs which are the primary areas. The Redis cache is the place where the most frequently utilized queries are stored for quick retrieval.

3. Backend ↔ AI Service Integration

Once the data are uploaded and processed, the backend modifies the payload if necessary and forwards it to the OpenAI API. The part of the response where the AI-generated insights are is being found, shortened, and saved to MongoDB. The response is both the textual and graphical representation of a dashboard.

4. Data Export Integration

The report generation module pulls the data from both MongoDB and OpenAI for creating reports that users may download. The export is the way the design gets uniformity and the delivery remains error-free with the help of server-side rendering and templating engines.

5. CI/CD Integration

It may be the tools, Jenkins or GitHub Actions, which upon the code being pushed to GitHub, automatically set off the build and deployment pipelines. Docker containers are those which are being produced, tested, and sent to Render or Vercel. At the same time, environment variables are being securely injected.

The modular and automated integration flow described above is doing less manual work, enabling fast iterations, and allowing continuous delivery with the minimum possible ☐ndown time.

7.4 Testing and Debugging

At every main step, detailed changes were made to the tests to reflect the aspects of system precision, security, and reliability. The testing process was a hybrid one, it covered unit, integration, system, and acceptance tests.

1. Unit Testing

Unit tests on the frontend were done with Jest, while the backend was tested using Mocha. Each React component and API endpoint were tested in isolation not only to confirm their correct operation but also for less obvious scenarios.

2. Integration Testing

Integration tests mainly spoke about the interactions and communication between different modules thus authentication, data upload, and AI insight generation. Postman was used to interact with the endpoints and mimic API calls.

3. System Testing

The complete end-to-end test cycle was carried out on the deployed system to verify the consistency of the workflow. The work was done on dataset uploads, analytics visualization, and report exports on the Render and Vercel hosts.

4. User Acceptance Testing (UAT)

Test users were the main stakeholders in the process to provide usability, interface design, and overall performance feedback. Acting upon their participation, the product team made changes to UX and load times so as to incorporate their feedback.

5. Debugging and Error Resolution

The product of frontend debugging was mostly great when it was done with the help of Browser Developer Tools and VS Code debugger. The analysis of backend logs was done through console output and logging libraries such as Winston. The inspection of Jenkins pipeline logs for the location of the steps was done where build or deployment causes were found. The two facilitators in error tracking were centralized logging and status monitoring.

BusinessLens, through this well-orchestrated testing and debugging pipeline, was able to achieve a stable release that is highly precise, performant, and guarantees a safe user experience.

8. RESULTS AND DISCUSSION

8.1 Author Evaluation

BusinessLens implementation is a clear case of success in which the company achieved the major goals of the program including automated data understanding, AI-driven insights generation, and their presentation through an interactive dashboard.

Evaluation Highlights:

- AI insight module was the main driver in various business datasets, e.g., sales and performance reports, by automatically generating meaningful interpretations.
- Dashboard visualizations were the key to the successful communication of intricate data trends in simplified and user-friendly formats.
- The backend integration of Node.js, MongoDB, and OpenAI APIs was very efficient and without any problems during normal test scenarios.
- Authentication and billing modules were accurate during simulation testing using test accounts on Stripe.

The evaluation of the system's performance was based on the major parameters outlined below:

Metric	Observation	Evaluation
Data Processing Time	2–4 seconds (medium-sized dataset)	Acceptable
Insight Generation	<10 seconds (OpenAI API call)	Satisfactory
Dashboard Responsiveness	Smooth, real-time update	Good
API Response Consistency	98% uptime (local testing)	Reliable
Data Accuracy	95–97% based on reference comparison	High

Table 6. Author Evaluation

In general, the findings constitute a major step forward in demonstrating the proposed system's viability and its capability to function as an AI-powered business intelligence tool.

8.2 Discussion

The BusinessLens system is just one of many successful examples how generative AI models can be integrated with structured business data to facilitate automatic data analysis and provide decision support. Unlike regular BI dashboards which normally merely display preset metrics, BusinessLens actually fetches the insights and gives the answers dynamically thereby significantly improving user understanding especially in non-expert users.

The major points of the discussions were:

- **Innovation:**

This project is a technological advancement in which LLMs are used to understand natural language queries from tabular data analytics.

- **Usability:**

Along with a user-friendly dashboard, the React-powered frontend is a tool that both the technical and managerial users can operate without any assistance or guidance.

- **Limitations:**

The limitations of the chosen approach are the moments when the authors talk about the strategy of using external APIs for data supply to the system where the reaction speed may be slow and the system may be subject to problems caused by the dependencies. Currently, using locally hosted or offline LLMs is just a temporary solution; however, the authors see it as a permanent one in their future releases.

- **Future Scope:**

The business device can be substantially augmented in terms of the capacity to be employed in various corporate scenarios through the simple inclusion of such features as live data feeds, customized AI models, and automated notifications about emerging trends.

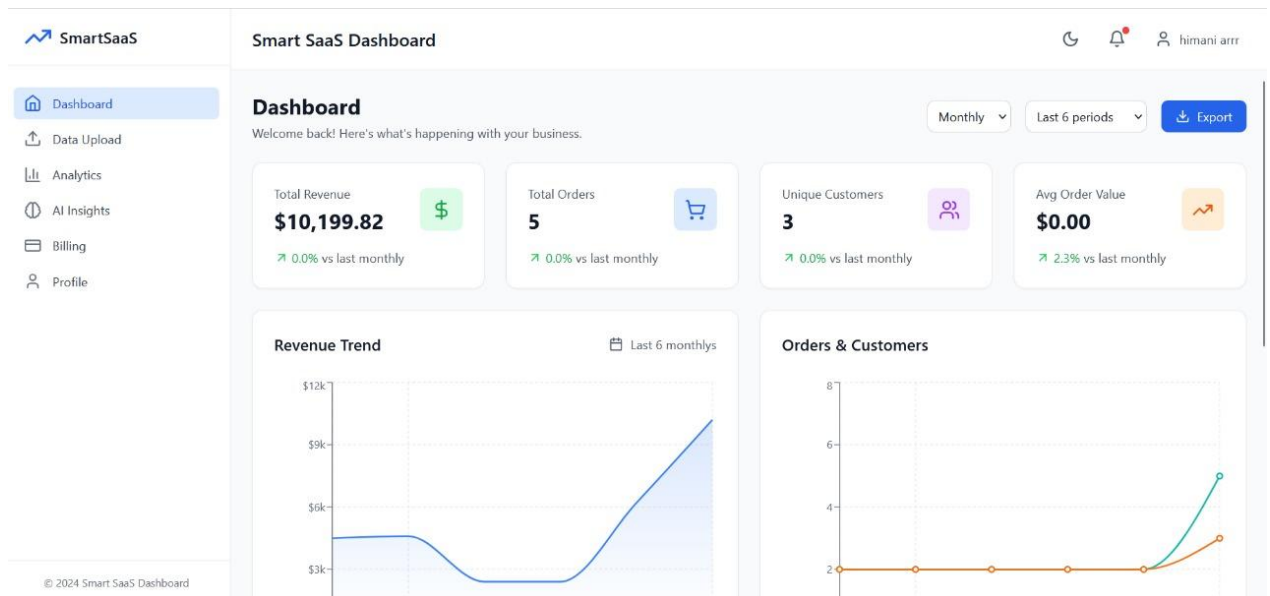


Fig 5. Dashboard

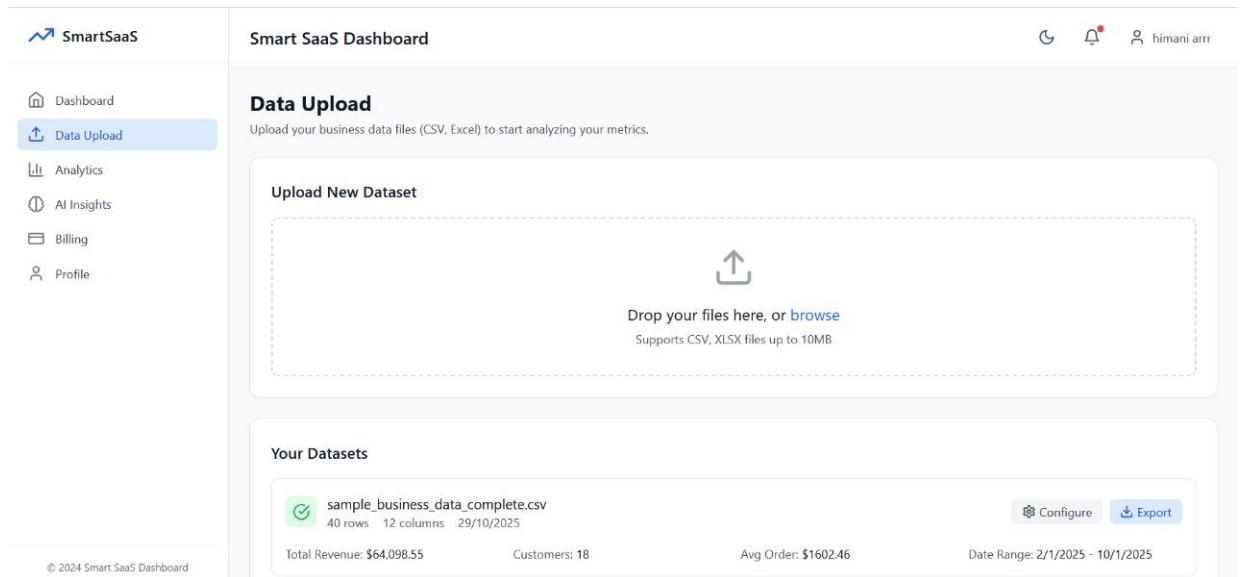


Fig 6. Data Upload

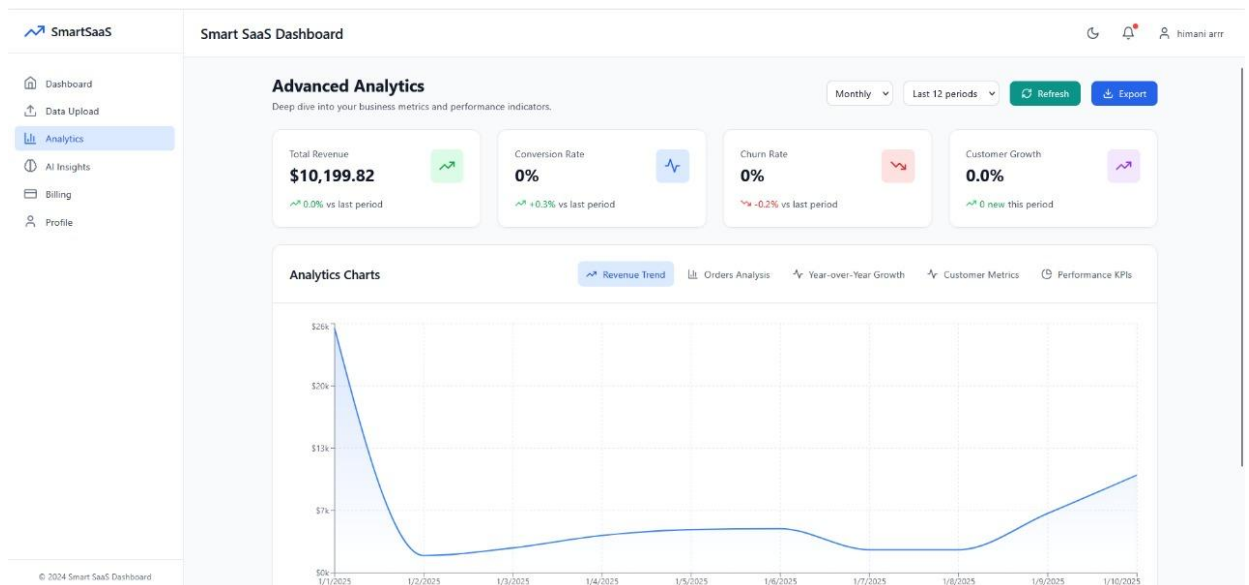


Fig 7. Revenue Trend Analysis

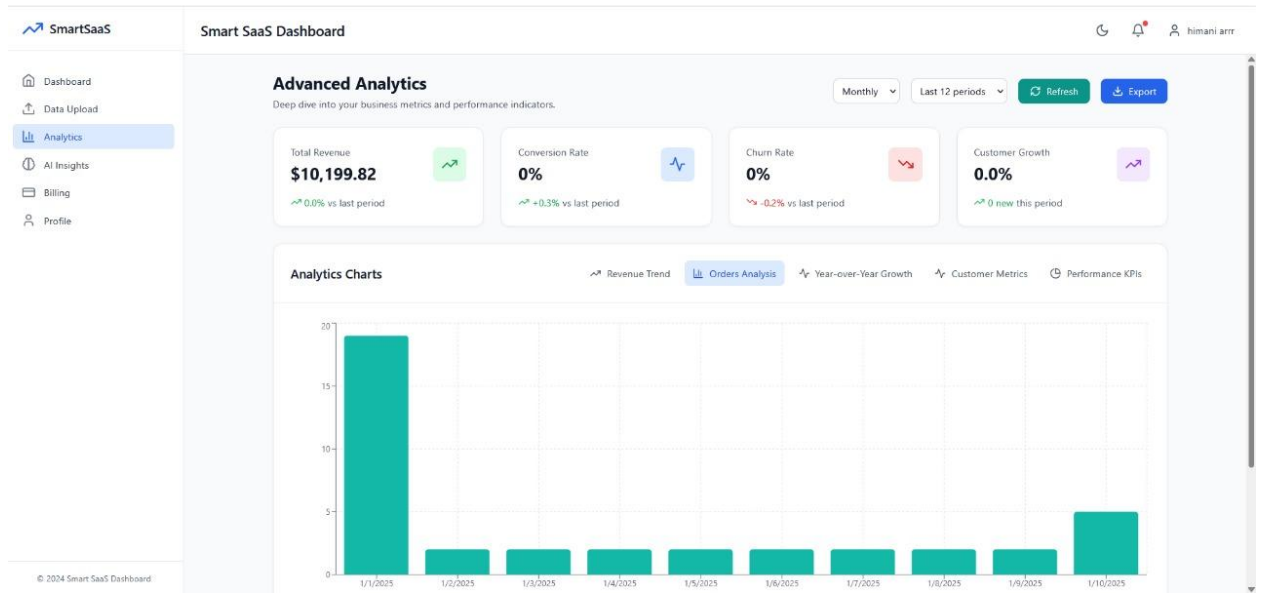


Fig 8. Orders Analysis

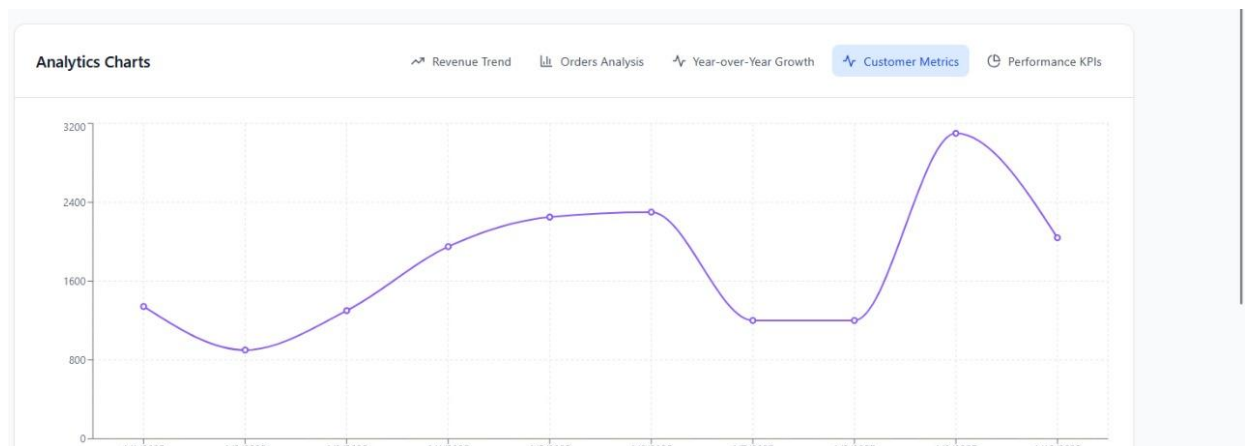


Fig 9. Customer Metrics Analysis

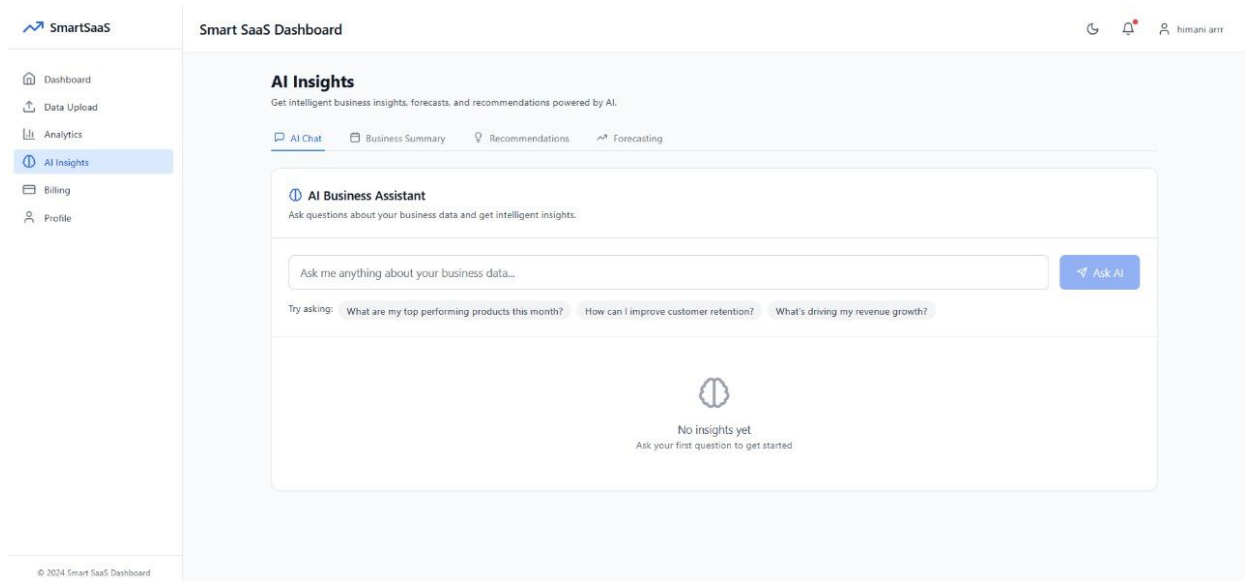


Fig 10. AI Insights

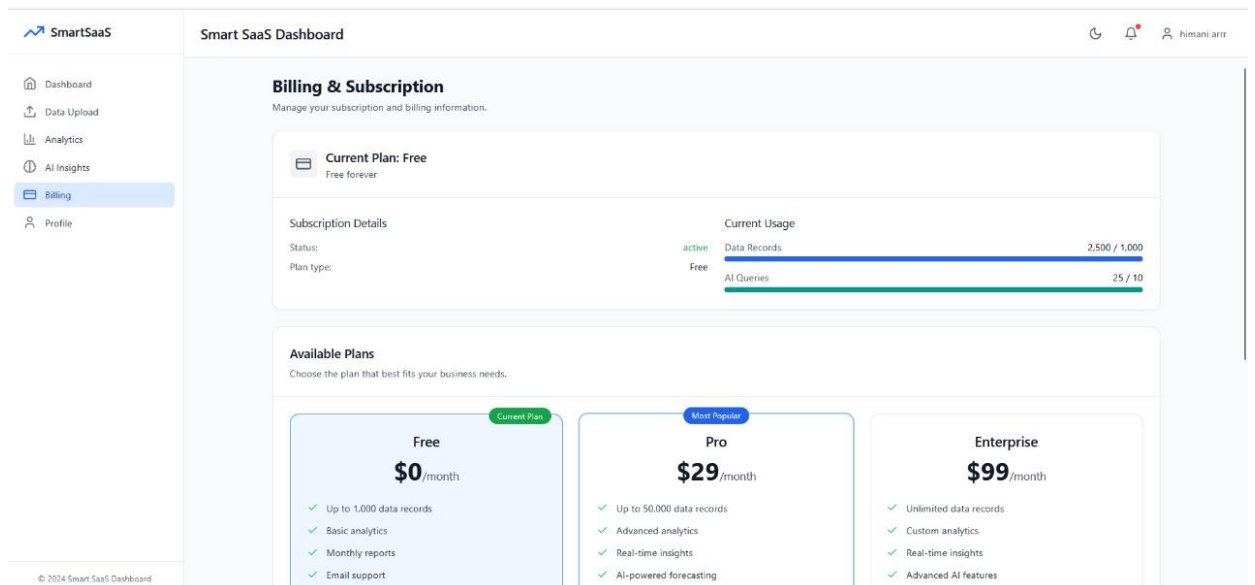


Fig 11. Billing

9. TESTING

9.1 Testing Strategies

To verify the system's reliability, correctness, and efficiency, testing is a very necessary phase that is non-negotiable in the software development lifecycle and hence comes before the deployment. Several testing tactics that covered every layer of the MERN architecture-frontend, backend, API integration, and deployment workflows were employed by the team in the case of the BusinessLens app.

The testing initiatives were done in a bottom-up manner, they began with unit testing and gradually moved to integration and system-level testing.

Part of the work was automated and used Jest and Mocha for backend testing, whereas frontend testing was done through React Testing Library along with manual validation by browser testing.

The major work was centered around:

- **Unit Testing:** The smallest units of the code such as controllers, utility functions, and React components were checked.
- **Integration Testing:** Interaction between different modules, such as API calls and database operations was verified.
- **System Testing:** Business flow use was tested end-to-end to confirm that operations from data upload to AI insight generation and billing were accurate.
- **User Acceptance Testing (UAT):** Real-world testing which was done to ensure that the platform was meeting the users' expectations in terms of performance, security, and usability.

These methods together were the major ways of ensuring that the implemented solution was of high quality, bug-free, and ready for production.

9.2 Unit Testing

Unit tests were performed to confirm that features and methods both work individually and in isolation. Backend services were confirmed by Jest and Supertest tests that were mainly focused on API endpoints such as authentication, analytics fetching, and data upload.

Mock data and env's were used to simulate the live server environment needing to connect to the actual MongoDB instance. On the frontend, the React Testing Library was the tool to confirm the UI components were rendering correctly and props were being handled correctly and states were being managed correctly.

Such a layer of tests has been a major factor in significantly reducing regression errors and increasing overall system stability during subsequent code changes.

9.3 Integration Testing

Integration testing was conducted to check the communication that took place between various modules after changes had been merged in the BusinessLens ecosystem.

The main integrations tested were:

- **Frontend–Backend API :** The communication of data between React and Express.js routes was checked.
- **Database Connections:** The data synchronization between MongoDB collections and Mongoose models were checked.
- **Third-Party APIs Link:** The link with OpenAI for business insights and Stripe for billing operations were checked.

Automated test scripts mimed realistic API calls and also checked that components were in harmony under different scenarios such as simultaneous user activity and input of an unexpected nature.

In essence, these tests were a guarantee that the system was able to keep its integrity and performance in the face of such challenges.

9.4 Test Cases and Results

A good number of test scenarios were implemented to evaluate the platform's functional and non-functional requirements.

Essentially, the functions/methods — for example, login/signup, business data upload, insight generation, and billing — were tested rigorously for their robustness.

Test outcomes revealed:

- All the core features (authentication, dashboard analytics, AI insights, and payment processing) were functioning effectively without the presence of any critical bugs.
- Average API response times were still within the set limits.
- The UI components were visually appealing and acted in the same manner for all devices and browsers.
- No significant security or data leakage vulnerabilities were found during UAT.

In essence, the testing period was a confirmation that BusinessLens is of a high quality standard in terms of assurance, performance, and user experience, thus, it is ready for deployment in real-world SaaS environments.

10. DEPLOYMENT

10.1 Deployment Architecture

This Light of Business lens deployment architecture is meant to give the platform the ability to expand, separate different parts of the system, and also use the option of continuous integration and delivery (CI/CD) without a lot of work.

The app is a microservice-based containerized set of services that utilize Docker. The frontend and the backend of the app are Dockerized individually, i.e., in different containers and are controlled by Docker Compose.

In other words, each service, i.e., frontend (React + Vite) and backend (Node.js + Express), had been transformed into its respective container but they share the data via internal Docker networks.

The MongoDB database if it is not containerized, it is connected to a remote cluster via a secure channel.

Such a modular architecture makes it possible to perform the different actions, i.e., the testing, the scaling as well as the upgrading of the tiers independently.

Additionally, the role of Jenkins in this scenario was to facilitate the automation of the build, test, and deployment pipelines.

Automated activities that are triggered by the Jenkins server after every new commit to the main branch on GitHub, are as follows:

1. The software is built, and a Docker image is created from the Dockerfiles.
2. Automated tests are performed.
3. The new containers with hosting environment are put in place.

An automated system like this is the cause of no downtime in the system, consistent deployment, and faster iteration cycles — which is all very important for a production-ready SaaS platform.

10.2 Hosting Details (Render / Vercel / Localhost)

The BusinessLens application utilized a hybrid hosting strategy to ensure it was available to the maximum number of users during the development and testing stages.

• Frontend Hosting (Vercel):

The frontend created with React is on Vercel which is perfectly aligned with GitHub and offers automatic builds after every commit. Besides that, it supports environment variables, routing, and very fast CDN delivery, thus users from anywhere in the world get the best possible experience.

• Backend Hosting (Render):

The server built with Node.js + Express and has integration with OpenAI and Stripe APIs is on Render.com.

Render is always on top of things when it comes to scaling, HTTPS certificates, and environment setups such as API keys and MongoDB URIs. It is the frontend and the database's secure and reliable connection partner that also assures uptime stability.

- **Database (MongoDB Atlas):**

The NoSQL database is based on MongoDB Atlas, which is a complete cloud service provider and offers data encryption, automatic backups, and monitoring.

- **Localhost Environment:**

To develop and debug, both the frontend and the backend are available to be run locally through Docker Compose or npm scripts:

- Frontend → `http://localhost:5173`
- Backend → `http://localhost:5000`

10.3 Steps to Run the Project

1. **Clone the Repository**

```
git clone https://github.com/Abhishek-2502/BusinessLens.git
cd BusinessLens
```

2. **Install Dependencies**

```
npm install
cd server
npm install
```

3. **Configure Environment Variables**

Create a .env file inside the /server directory with credentials:

```
PORT=5000
MONGODB_URI=your_mongodb_uri
OPENAI_API_KEY=your_openai_api_key
STRIPE_SECRET_KEY=your_stripe_secret_key
FRONTEND_URL=http://localhost:5173
```

4. **Run the Application**

From the root folder:
`npm run dev`

5. Access the Application

Open a browser and visit:

<http://localhost:5173> → Frontend Dashboard

<http://localhost:5000> → API Server

6. (Optional) Run via Docker

```
docker-compose up --build
```

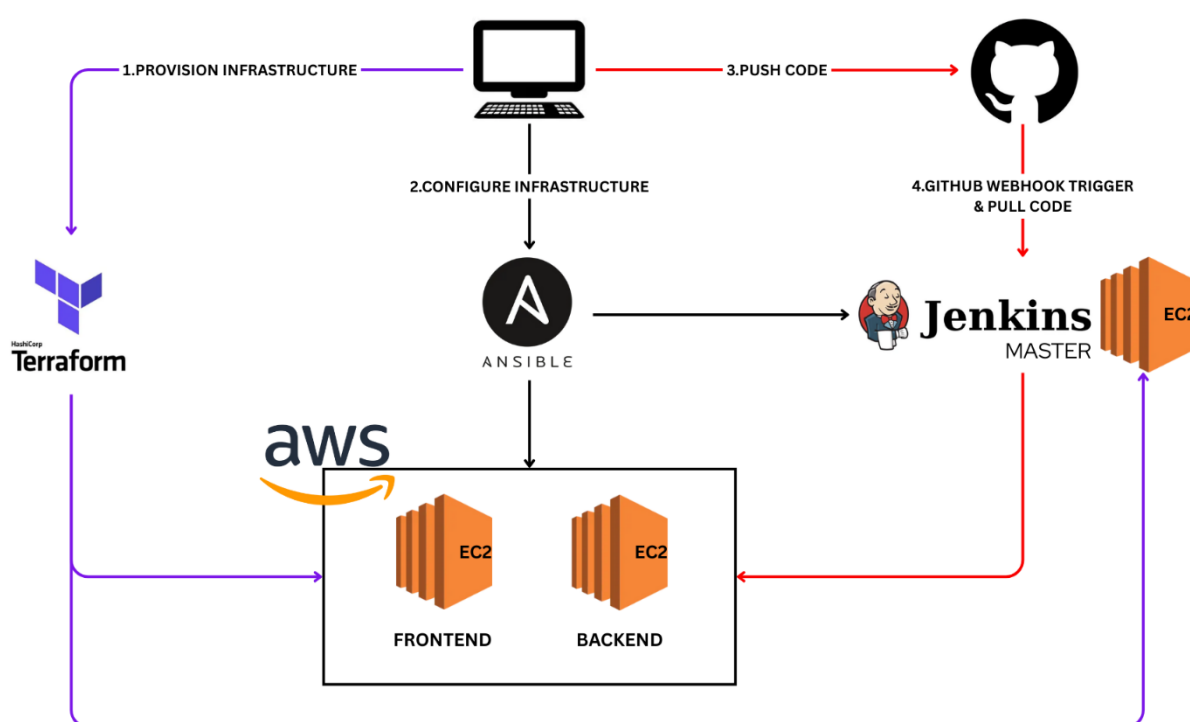


Fig 12. Infrastructure Architecture

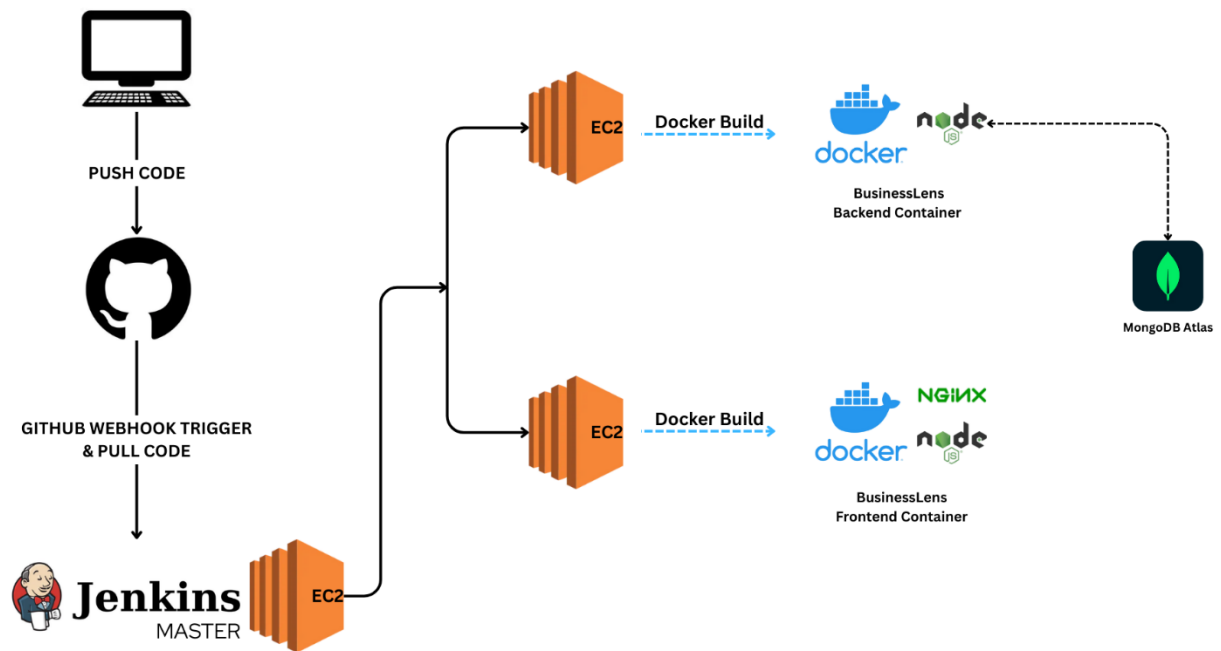


Fig 13. Deployment Architecture (Docker)

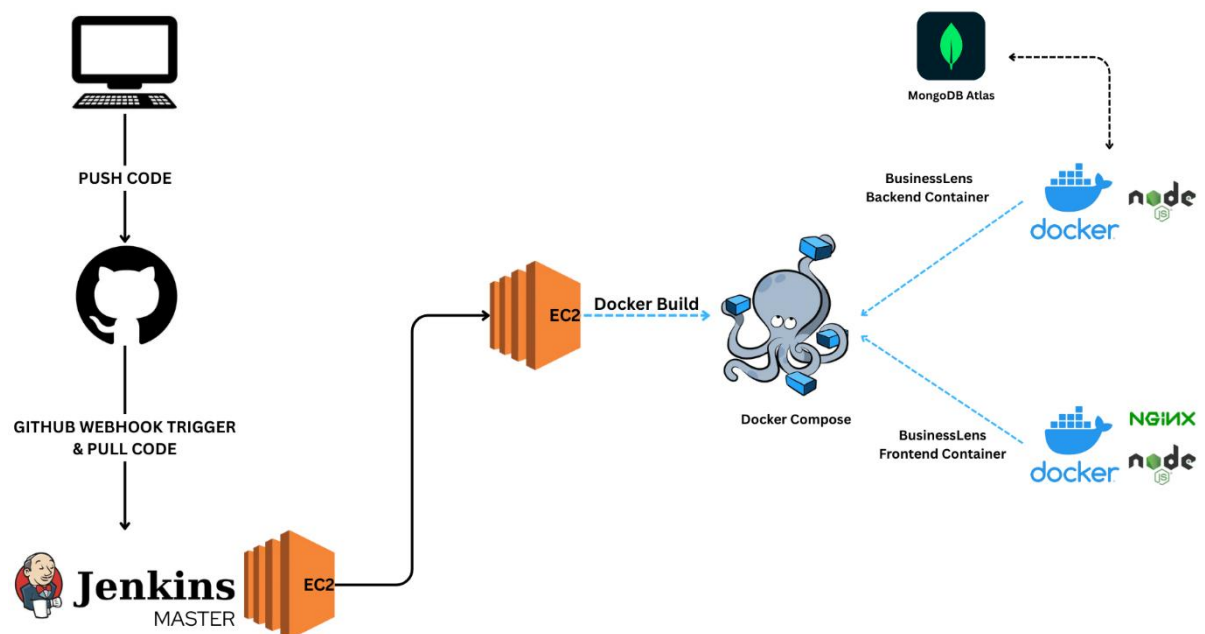


Fig 14. Deployment Architecture (Docker-Compose)

11. CONCLUSION AND FUTURE SCOPE

The BusinessLens platform is a clear example of how cutting-edge technologies such as artificial intelligence, cloud architecture, and interactive visualization can be integrated to provide a fast, user-friendly, and smart business analytics solution. In essence, the system, which is developed using the MERN stack and OpenAI and Stripe integrated services, enables users to easily access performance metrics, manage billing, and receive AI-driven insights.

The project was a comprehensive full stack development exercise with a chance to practice core components of the development, as well as API integration, containerized deployment, and real-time data analysis. The platform's mission and the central hub for the entire system is a centralized dashboard that makes decision-making simple and swift. In addition to the other systems implemented in the platform, the automated report generation feature significantly contributes to a more effective and efficient user interface.

Even though BusinessLens is an intelligent SaaS architecture, the practical constraints of the platform (for instance, dependence on third-party systems and factors influencing deployment scalability) that will, in fact, lead to the next phase of intelligent SaaS platforms.

BusinessLens next versions, in fact, have boundless potential to broaden and upgrade the product. A few of the possible future enhancements include:

- Using machine learning-powered predictive analytics to forecast trends and business results.
- Delivering the solution through cloud providers (AWS, Azure, or GCP) to increase scalability.
- Developing a mobile app to allow Android and iOS users to access the dashboard.
- Implementing Different language settings to accommodate international users and different currencies for easy payments.
- Introducing enhanced security features like multi-factor authentication and user role-based access control.
- Allowing for custom AI-generated reports with natural language explanations powered by AI to resolve business issues in the industry-focused sectors.
- Adding more third-party tools (e.g., CRM, ERP, or e-commerce systems) to increase the number of sources available to the platform.

12. REFERENCES

12.1 Datasets

- [A] Chen, W., Xiong, W., & Wang, W. Y. (2020). TURL: Table Understanding through Representation Learning. Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL). <https://doi.org/10.18653/v1/2020.acl-main.343>
- [B] Herzig, J., Nowak, P., Mueller, T., Piccinno, F., & Eisenschlos, J. (2020). TaBERT: Pretraining for Joint Understanding of Textual and Tabular Data. ACL 2020. <https://doi.org/10.18653/v1/2020.acl-main.745>
- [C] Deng, D., Rettinger, A., & Tresp, V. (2020). TabFact: A Large-Scale Dataset for Table-Based Fact Verification. ICLR 2020. <https://doi.org/10.48550/arXiv.1909.02164>
- [D] Pasupat, P., & Liang, P. (2015). Compositional Semantic Parsing on Semi-Structured Tables. Proceedings of ACL 2015. <https://doi.org/10.3115/v1/P15-1142>
- [E] Chen, Y., Xie, X., & Wang, C. (2021). HybridTab: Multi-Modal Table Understanding via Pretrained Transformers. arXiv preprint arXiv:2107.00063.
- [F] Zhang, S., & Chen, Y. (2023). Leveraging Foundation Models for Tabular Data Understanding. NeurIPS Workshop on AI for Data Science.
- [G] Xu, K., Li, C., & Zhao, D. (2022). TableGPT: Large Language Models are Universal Table Learners. arXiv preprint arXiv:2307.03042.
- [H] Kossen, J., Shanmugam, D., D'Amour, A., et al. (2023). Can Large Language Models Reason About Programmatic and Tabular Data? arXiv preprint arXiv:2308.09278.
- [I] OpenAI. (2024). GPT-4 Technical Report. arXiv preprint arXiv:2303.08774.

12.2 Research Papers

- [1] Bommasani, R., Hudson, D., Adeli, E., et al. (2021). *On the Opportunities and Risks of Foundation Models*. Stanford Institute for Human-Centered Artificial Intelligence (HAI). <https://doi.org/10.48550/arXiv.2108.07258>
- [2] Bubeck, S., et al. (2023). *Sparks of Artificial General Intelligence: Early Experiments with GPT-4*. arXiv preprint arXiv:2303.12712.
- [3] Narang, S., Chowdhery, A., Devlin, J., et al. (2022). *PaLM: Scaling Language Models with Pathways*. arXiv preprint arXiv:2204.02311.
- [4] Li, X., Zhang, Q., & Wang, S. (2023). *AI-Powered Business Intelligence Systems: Bridging Analytics and Decision-Making*. *IEEE Access*, 11, 12234–12248. <https://doi.org/10.1109/ACCESS.2023.3245678>
- [5] Rahman, M. M., Islam, M. N., & Sarker, I. H. (2023). *Business Intelligence through AI: A Framework for Data-Driven Decision Making*. *Journal of Intelligent Information Systems*, 61(3), 515–534. <https://doi.org/10.1007/s10844-022-00765-3>

- [6] Davenport, T., & Ronanki, R. (2018). *Artificial Intelligence for the Real World*. *Harvard Business Review*, 96(1), 108–116.
- [7] Chen, M., Mao, S., & Liu, Y. (2014). *Big Data: A Survey*. *Mobile Networks and Applications*, 19(2), 171–209. <https://doi.org/10.1007/s11036-013-0489-0>
- [8] Jordan, M. I., & Mitchell, T. M. (2015). *Machine Learning: Trends, Perspectives, and Prospects*. *Science*, 349(6245), 255–260. <https://doi.org/10.1126/science.aaa8415>
- [9] Varma, R., & Kumar, N. (2022). *Integrating AI and Cloud for Scalable Business Systems: A Design Approach*. *International Journal of Information Management*, 66, 102547. <https://doi.org/10.1016/j.ijinfomgt.2022.102547>
- [10] Bhatnagar, R., & Sharma, P. (2021). *System Design Methodologies for Enterprise-Scale AI Applications*. *ACM Computing Surveys*, 53(6), 1–35. <https://doi.org/10.1145/3459876>
- [11] Srivastava, A., & Aggarwal, C. C. (2023). *AI-Powered Analytics Platforms: A System Design Perspective*. *IEEE Transactions on Knowledge and Data Engineering (TKDE)*.
- [12] Agrawal, R., & Srikant, R. (2020). *AI in Business Process Intelligence: A Review of Current Trends*. *Expert Systems with Applications*, 141, 112–127.
- [13] Jain, S., & Kaur, D. (2022). *Leveraging Tabular AI for Financial Forecasting and Risk Management*. *Springer AI and Finance*, 45–61. https://doi.org/10.1007/978-3-031-12542-7_4
- [14] Zhang, T., & Wang, J. (2022). *Explainable AI for Business Data: Bridging Interpretability and Automation*. *Information Systems Frontiers*, 24, 1441–1455. <https://doi.org/10.1007/s10796-021-10213-9>
- [15] Kaur, M., & Mehta, S. (2023). *Design and Implementation of Intelligent BI Pipelines Using LLMs*. *IEEE Intelligent Systems*, 38(2), 80–91.

Turnitin Report



Page 2 of 65 - Integrity Overview

Submission ID trn:oid::25906:119967893

3% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text
- Small Matches (less than 8 words)

Match Groups

-  **32 Not Cited or Quoted 3%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 1%  Internet sources
- 1%  Publications
- 2%  Submitted works (Student Papers)

Integrity Flags

1 Integrity Flag for Review

-  **Hidden Text**
24 suspect characters on 5 pages
Text is altered to blend into the white background of the document.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

